

OpenBlock: A Full-Stack Adaptive Tile-Matching Platform with Reinforcement Learning and Programmatic Content Generation

Technical Report — v1.1

Authors: OpenBlock Contributors

Status: Complete Draft

Last Updated: 2026-07-01

Project URL: <https://github.com/btbujiangjun/openblock>

License: MIT

Abstract

OpenBlock is an open-source research platform for adaptive, personalized puzzle gameplay that unifies four co-evolving pillars—a tile-matching game engine, an adaptive spawn AI, a self-play reinforcement learning (RL) agent, and a non-intrusive monetization framework—under a single real-time player profile. At its core lies a **dual-track spawn architecture**: a deterministic rule-based heuristic engine (`SpawnPolicyRules`) provides always-available content generation with sub-5ms latency, while an optional Transformer-based neural generator (`SpawnPolicyNet` , ~317K parameters) learns the conditional distribution $P(s_1, s_2, s_3 \mid \text{board, profile, history})$ from real player replay data. Both tracks feed through a unified **nine-layer generation pipeline** terminating in depth-first search (DFS) sequential feasibility verification—guaranteeing by construction that every delivered dock is fully placeable. The RL agent is trained via Proximal Policy Optimization (PPO) with Generalized Advantage Estimation (GAE) and seven auxiliary supervision heads that inject dense per-step gradient signals, including a novel **per-shape placeability head** (v13) that quantifies whether long-bar polyominoes (1×4 , 1×5) remain viable on the current board. Training on 230,000+ self-play episodes achieves a 35.6% win rate with a median score of 4,200. Empirical analysis identifies the long-bar bottleneck as the primary late-game failure mode: at $\geq 70\%$ board fill, 33–56% of 1×4 and 1×5 pieces have zero legal placements. The full platform is configuration-driven via `game_rules.json`, browser-first (playable without backend), and validated by cross-language (JavaScript $\leftrightarrow$ Python) contract tests.

Keywords: tile-matching game, adaptive difficulty, reinforcement learning, procedural content generation, player modeling, polyomino puzzle, self-play, PPO, GAE, dual-track architecture, nine-layer pipeline

1. Introduction

1.1 Background and Motivation

The puzzle tile-matching genre has experienced explosive growth since 2020. Block Blast, the category leader, surpassed 300 million monthly active users (MAU) as of late 2024, built on a deceptively simple core loop: an 8×8 grid, a set of polyomino pieces, and a single drag-and-drop placement action per turn. In 2025, Color Block Jam introduced color-matching mechanics and pre-designed puzzle levels, shifting the genre paradigm from “player adapts to random pieces” toward “the system adapts to the player.”

Despite this commercial success, the technical architecture underlying these games remains largely opaque. Content generation—the decision of which pieces to present at each turn—is treated as a proprietary black box, with no published descriptions of the algorithms involved. Player state estimation, when it exists, is rudimentary (typically limited to simple win/loss ratios). Difficulty adaptation, if present, operates on coarse time scales (session-level or daily). There exists no standard open-source platform where researchers can experiment with adaptive difficulty algorithms, reinforcement learning for puzzle placement, or explainable content generation in a realistic, production-quality tile-matching environment.

OpenBlock addresses this gap. It is designed explicitly as a **research platform** rather than a commercial product. Every design decision is transparent. Every spawn decision carries a diagnostic snapshot recording which signals drove the choice, which constraints were applied, and why each candidate was selected or rejected. Every algorithm parameter is externalized in a version-controlled configuration file. The platform is MIT-licensed and fully open source.

1.2 Design Philosophy

OpenBlock is built around four principles that distinguish it from both commercial alternatives and academic prototypes:

Principle 1: Four pillars, one player profile. The game engine, adaptive spawn AI, RL training system, and monetization framework all read from and contribute to a single, shared `PlayerProfile` data structure. Spawn difficulty, ad timing, and RL state encoding are grounded in a consistent, real-time estimate of the player’s current skill, flow state, and frustration level. There is no fragmentation between “the game’s model of the player” and “the monetization system’s model of the player.”

Principle 2: Offline-first by design. The entire core game loop—including the spawn engine with its nine-layer pipeline, player profiling, difficulty decisions, and the constructive pre-scan—runs entirely in the browser with zero network dependency. Backend services (RL training, neural spawn inference, player analytics, auth) are optional enhancements, not requirements. The game is fully playable by opening `index.html` in a browser with no server running.

Principle 3: Configuration-driven, never code-driven. Every numerical parameter, threshold, weight, and feature dimension is externalized in `shared/game_rules.json` or in environment variables. A project-wide “no magic numbers” rule is enforced: algorithm code that hardcodes a numerical constant is rejected at code review. This makes the system auditable, reproducible across deployments, and safe for A/B experimentation—a configuration change never requires a code redeployment.

Principle 4: Explainable by construction. The spawn engine records a complete diagnostic snapshot at every step. The `PlayerInsightPanel` renders this as a human-readable decision trace. The `DecisionFlowViz` exposes the 10-signal stress chain with per-signal attribution. The `spawn-signal-explorer.html` interactive tool lets developers step through the pipeline signal by signal. This is not a bolt-on explainability layer—it is a natural consequence of the architecture: since every weight, constraint, and decision is explicit, explaining it is simply a matter of rendering the internal state.

1.3 Contributions

This technical report presents the following contributions:

1. **A dual-track spawn architecture** (Chapter 6) that combines a deterministic rule-based engine (`SpawnPolicyRules`) with an optional Transformer-based neural generator (`SpawnPolicyNet`) under a unified constraint-validation gate and automatic fallback mechanism. This architecture achieves production reliability (the default rule track is always available) while enabling learned improvements (the neural track can discover distributional patterns inaccessible to hand-tuned weights).
2. **A nine-layer content generation pipeline** (`generateDockShapes`) that decomposes the monolithic spawn problem into sequentially composed, independently verifiable stages: input assembly → board perception → score construction → priority selection → weighted completion → constraint verification → injection optimization → output delivery → color display. Each layer addresses a specific, well-defined sub-problem with its own configuration section, making the pipeline auditable, testable, and incrementally improvable.
3. **A real-time player profiling system** (Chapter 5) that estimates player skill via exponentially-weighted moving averages with adaptive decay rates, detects flow deviation from optimal challenge-skill balance, tracks frustration on a multi-threshold escalation scale, computes momentum from sliding window analysis, and projects each player onto a two-dimensional 5×5 lifecycle-by-maturity differentiation grid. All computations run in-browser with sub-millisecond latency.
4. **A self-play RL agent** (Chapter 8) trained with PPO and GAE, augmented by seven auxiliary supervision heads that provide dense per-step gradient signals independent of sparse Monte Carlo returns. The v13 extension introduces per-shape placeability prediction—explicitly quantifying whether long-bar shapes (1×4, 1×5) remain placeable on the current board—as an auxiliary target, directly addressing the empirically identified primary bottleneck for late-game survival.
5. **A full-stack open-source implementation** with contract-first cross-language consistency (JavaScript ↔ Python), comprehensive test infrastructure (unit, lint, build, algorithm regression, cross-language equivalence), a configuration-driven architecture suitable for both research and production deployment, and cross-platform support (Web, WeChat Mini Program, iOS/Android via Capacitor, Cocos Creator).
6. **Empirical characterization** (Chapter 13) of the long-bar bottleneck—the observation that 1×4 and 1×5 polyominoes lose 33–56% of their legal positions at ≥70% board fill rate, making them the primary cause of late-game death—motivating and validating the v13 architecture change.

1.4 Report Organization

The remainder of this report follows a bottom-up structure. §2 formalizes the game mechanics and the three sub-problems (spawn, placement, difficulty modulation). §3 presents the system architecture. §4 covers the player profiling engine. §5 describes the spawn engine—the system’s algorithmic core. §6 covers scoring, placement quality evaluation, and the feedback closed loop. §7 presents the RL agent architecture, training methodology, and auxiliary supervision design. §8 describes the neural spawn generation model (`SpawnPolicyNet`). §9 covers the spawn parameter tuning system (`SpawnParamTuner`). §10 briefly covers the monetization framework. §11 describes engineering infrastructure. §12 presents empirical evaluation results. §13 surveys related work. §14 discusses limitations and future directions. §15 is an appendix with glossary, configuration reference, API reference, and reproduction checklist.

2. Game Overview and Problem Formulation

2.1 Core Mechanics

OpenBlock’s core gameplay is a single-player tile-matching puzzle on an 8×8 grid. The state space, action space, and transition dynamics are:

Grid state. The board is an 8×8 matrix $B \in \{-1, 0, 1, \dots, K\}^{64}$ where $B_{ij} = -1$ indicates an empty cell, and $B_{ij} \in [0, K]$ indicates a cell occupied by a block of color B_{ij} .

Shape catalog. 28 polyomino shapes are organized into 7 categories:

Category	Count	Examples	Cell Count Range
Lines	8	$1 \times 2, 2 \times 1, 1 \times 3, 3 \times 1, 1 \times 4, 4 \times 1, 1 \times 5, 5 \times 1$	2–5
Rects	2	$2 \times 3, 3 \times 2$	6
Squares	2	$2 \times 2, 3 \times 3$	4, 9
T-shapes	4	T-up, T-down, T-left, T-right	4–5
Z-shapes	8	diagonal-2a/b, diagonal-3a/b/c/d, zigzag	2–5
L-shapes	12	L3-a/b/c/d, L4 variants	3–5
J-shapes	4	J-1/2/3/4	3–5

Each shape is defined by a binary matrix of its occupied cells. Shapes have fixed orientation (no rotation in the standard game mode). The shape catalog is defined in `shared/shapes.json` and loaded identically by the JavaScript frontend and Python simulator.

Dock. At each step, the player is presented with exactly three candidate shapes (the “dock”): $D_t = \{s_1, s_2, s_3\}$, where each s_k is selected from the 28-shape catalog. The three shapes must be distinct.

Placement action. The player selects one dock block and places it at position (gx, gy) on the grid, where $0 \leq gx < 8 - w, 0 \leq gy < 8 - h$ (with w, h being the shape’s bounding box dimensions). The placement is valid if and only if no cell of the placed shape overlaps an occupied grid cell:

$$valid(B, s, gx, gy) \iff \forall (i, j) \in cells(s) : B[gy + i][gx + j] = -1$$

Line clearing. After placement, the game checks all rows and columns for completion. A row r is cleared if $\forall c : B[r][c] \geq 0$ (all cells occupied). A column c is cleared if $\forall r : B[r][c] \geq 0$. Cleared cells are removed (set to -1), and the player earns points.

Termination. The game ends when none of the three dock blocks have a legal placement position: $\forall s_k \in D_t : legalPositions(B, s_k) = \emptyset$.

2.2 The Three-Layer Decision Pipeline

The game’s AI systems are organized into three conceptual layers, each with a distinct responsibility and a well-defined interface to adjacent layers:

```
Layer 1 – Perception (Board → 17 Signals)
  Input: Board state B_t
  Process: Board topology analysis → feature extraction → signal computation
  Output: 17-signal vector → stress ∈ [-0.2, 0.85]
  Key modules: boardTopology.js, spatialPlanning.js, realtimeStrategy.js

Layer 2 – Decision (Signals → Intent)
  Input: stress, player profile p_t, arc stage, session context
  Process: Stress classification → spawnIntent resolution → difficulty targeting
  Output: spawnIntent ∈ {warm_run, relief, engage, ..., maintain}, d* ∈ [0,1]
  Key modules: adaptiveSpawn.js, intentResolver.js, difficultyRelativity.js
```

```

Layer 3 – Execution (Intent → Spawn)
  Input: Board  $B_t$ , spawnIntent,  $d^*$ , player profile  $\pi_t$ , spawn context  $ctx_t$ 
  Process: 9-layer pipeline → constructive pre-scan → two-stage weighted fill →
  constraint gate
  Output: Dock triplet  $(s_1, s_2, s_3)$ , diagnostic snapshot
  Key modules: blockSpawn.js (generateDockShapes), spawnModel.js (neural
  alternative)

```

This decomposition is critical because it separates concerns that would otherwise create combinatorial complexity. The perception layer analyzes *what is on the board*. The decision layer determines *what kind of experience the player needs*. The execution layer translates that intent into *concrete content*. Each layer can be tested, tuned, and replaced independently.

2.3 Formal Problem Statements

The system addresses three formally distinct sub-problems:

Problem 1: The Spawn Problem. Given:

- Board state B_t (8×8 grid with occupied/empty cells and color assignments)
- Player profile $\pi_t = (\text{skill}, \text{flow}, \text{frustration}, \text{momentum}, \text{lifecycle}, \text{maturity})$
- Spawn context ctx_t (recent history $H_{t-3:t-1}$, difficulty target d^* , spawn intent I , arc phase)

Find a dock triplet (s_1, s_2, s_3) from the 28-shape catalog $\mathcal{S}$ such that the following constraints and objectives are satisfied:

Hard constraints (violation → rejection, retry up to 22 times):

$$C1(\text{Uniqueshapes}) : s_1 \neq s_2 \neq s_3$$

$$C2(\text{Mobility}) : \sum_{k=1}^3 |\text{legalPositions}(B_t, s_k)| \geq \text{minMobilityTarget}$$

$$C3(\text{Sequential feasibility}) : \exists \text{ordering } \sigma \text{ of } \{1, 2, 3\} \text{ s.t. } DFS(B_t, s_{\sigma(1)}, s_{\sigma(2)}, s_{\sigma(3)}) > 0$$

Soft objectives (weighted maximization):

$$O1(\text{Clearpotential}) : \max \mathbb{E}[\text{linescleared}]$$

$$O2(\text{Difficultyalignment}) : \min |SCD(s_1, s_2, s_3, B_t) - d^*|$$

$$O3(\text{Diversity}) : \max H(\text{category}(\{s_1, s_2, s_3\}))$$

$$O4(\text{Delight}) : P(\text{multi} - \text{clear} \vee \text{icon} - \text{bonus} \vee \text{perfect} - \text{clear}) \cdot w_{\text{delight}}$$

Problem 2: The Placement Problem (RL agent's domain). Formulated as a finite-horizon Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T)$$

where:

- $\mathcal{S} = \mathbb{R}^{204}$: the 204-dimensional state feature encoding (§7.2)
- $\mathcal{A}_t$: the set of legal placement actions at time t (variable cardinality, $\leq 3 \times 64 = 192$)
- $\mathcal{P}$: deterministic transition function (placement → clear → new dock spawn)
- $\mathcal{R}$: reward function (Equation 8, §7.1)
- $\gamma = 0.99$: discount factor
- T : episode terminates when no dock block has a legal placement or score $\geq$ win threshold

The agent's objective is to find a policy $\pi^*(a|s)$ that maximizes expected cumulative discounted reward:

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

Problem 3: The Difficulty Modulation Problem. Maintain each player in their optimal Flow channel—where perceived challenge approximately matches perceived skill:

$$F(t) = \left| \frac{\text{boardPressure}(B_t)}{\max(0.05, \text{skill}(\pi_t))} - 1 \right| \leq \epsilon_{flow}$$

by modulating the spawn step difficulty target $d^* \in [0, 1]$ on a per-step basis, with additional modulation from lifecycle stage, session arc phase, momentum, and recent performance trajectory.

2.4 Constraint Verification Details

The sequential feasibility constraint (C3) is the most computationally expensive and algorithmically important. It is verified by a bounded depth-first search:

Algorithm: Sequential Feasibility DFS

```
function count_sequential_solution_leaves(board, dock, leaf_cap=1, node_budget=200):
    remaining ← {blocks in dock not yet placed}
    depth ← |remaining|
    if depth = 0: return 1

    nodes ← 0

    function dfs(placed_count):
        nonlocal nodes
        if placed_count = depth: return 1          # all blocks placed successfully
        if nodes ≥ node_budget: return 0           # budget exhausted →
        conservatively infeasible

        legal ← get_legal_actions(board, remaining - placed)
        if legal = ∅: return 0                      # dead end

        subtotal ← 0
        saved_state ← save_state(board)
        for each action a ∈ legal:
            if nodes ≥ node_budget or subtotal ≥ leaf_cap: break
            nodes ← nodes + 1
            apply_action(board, a)
            subtotal ← subtotal + dfs(placed_count + 1)
            restore_state(board, saved_state)

        return min(subtotal, leaf_cap)

    # Set search_mode flags to skip expensive operations during DFS
    board._search_mode ← True                      # skip eval-feedback O(|A|) computation
    board._skip_dock_respawn ← True                # skip constructive respawn (unused in DFS)
    try:
        leaves ← dfs(0)
    finally:
        board._search_mode ← False
        board._skip_dock_respawn ← False
        restore_state(board, saved_root)

    return min(leaves, leaf_cap)

function check_sequential_feasibility(board, dock, node_budget=200) → bool:
    return count_sequential_solution_leaves(board, dock, leaf_cap=1, node_budget) >
0
```

The budget of 200 nodes provides a conservative yet practical bound: if a feasible ordering exists, the DFS typically finds it within the first 50 nodes. Setting `leaf_cap = 1` makes this a binary feasibility check rather than a solution counter, minimizing wasted exploration. The `_search_mode` and `_skip_dock_respawn` flags disable expensive operations that are irrelevant during the feasibility probe (evaluation feedback computation costs $O(|A|)$ and constructive dock respawning costs $\sim 0.34s$ per call), reducing DFS overhead by over 90%.

3. System Architecture

3.1 Four-Pillar Overview

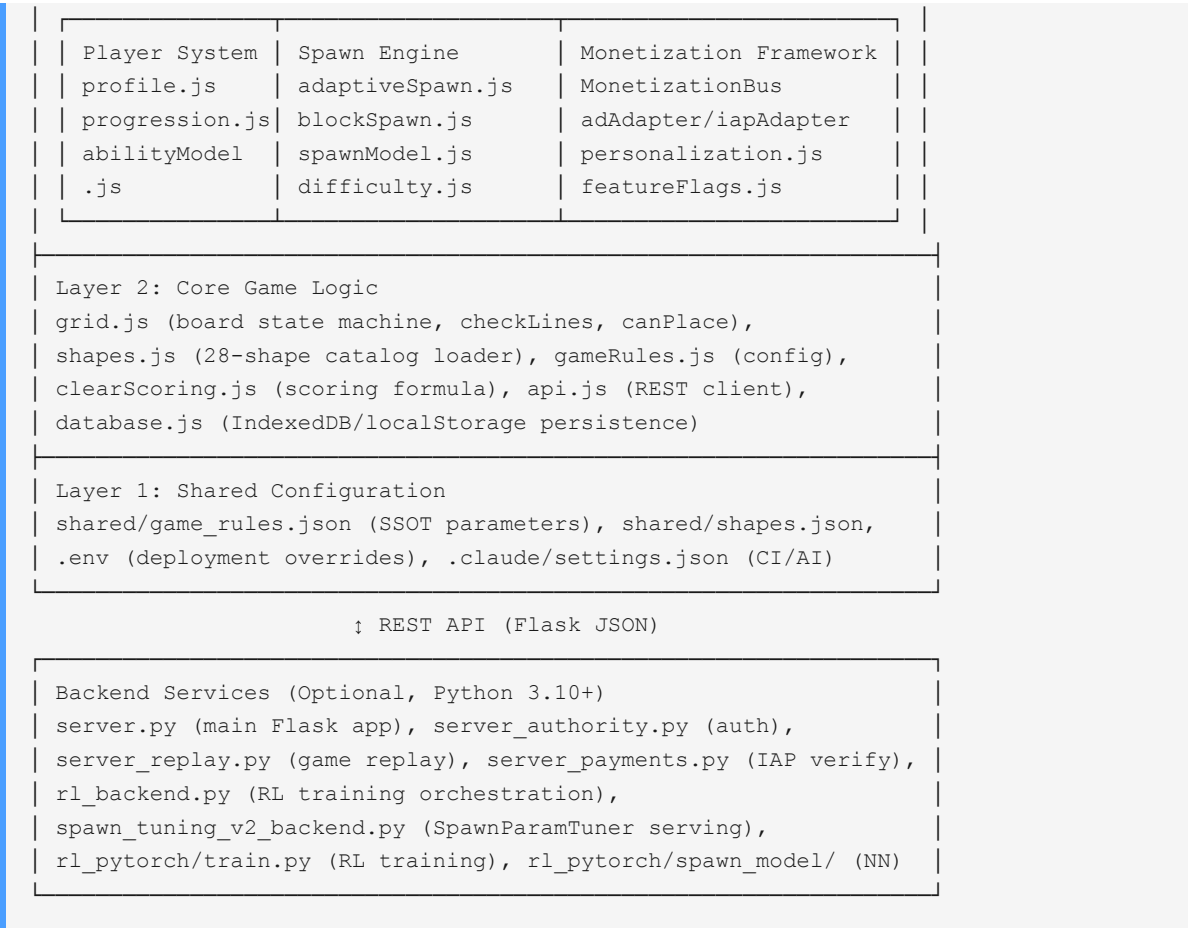
OpenBlock’s architecture is organized around four co-equal pillars that share a single `PlayerProfile` data structure:

Pillar	Responsibility	Runtime Location	Primary Source Files
 Game Engine	Grid state machine, placement validation, line clearing, scoring, canvas rendering	Browser	<code>grid.js</code> , <code>game.js</code> , <code>clearScoring.js</code> , <code>renderer.js</code>
 Adaptive Spawn AI	Content generation, difficulty modulation, player state estimation, between-game arc	Browser (primary) / Server (neural)	<code>adaptiveSpawn.js</code> , <code>blockSpawn.js</code> , <code>playerProfile.js</code> , <code>difficultyRelativity.js</code>
 RL Training	Self-play agent training, policy/value network optimization, evaluation gate	Python (server)	<code>rl_pytorch/train.py</code> , <code>rl_pytorch/model.py</code> , <code>rl_pytorch/simulator.py</code>
 Monetization	Ad timing, IAP offer scheduling, whale segmentation, LTV prediction	Browser + Server	<code>monetization/index.js</code> , <code>personalization.js</code> , <code>ltvPredictor.js</code>

Key architectural boundary: The RL agent operates on a completely separate code path from human players. The agent uses `rl_pytorch/simulator.py`—a numpy-accelerated, Numba-JIT-compiled port of the browser game engine—and its placement decisions are made by the neural policy, not by a human. This separation means that RL improvements do not affect human gameplay until explicitly deployed through the evaluation pipeline, and conversely, spawn algorithm changes for human players do not require RL retraining (though they may benefit from it).

3.2 Five-Layer Technical Stack

Layer 5: Presentation & Diagnostics
<code>renderer.js</code> , <code>playerInsightPanel.js</code> , <code>rlPanel.js</code> , <code>monPanel.js</code> , <code>spawnModelPanel.js</code> , <code>hintEngine.js</code> , <code>replayUI.js</code> , <code>DecisionFlowViz.js</code> , <code>spawn-signal-explorer.html</code>
Layer 4: Application Orchestration
<code>game.js</code> (main controller, <code>step()</code> → <code>clear()</code> → <code>spawn()</code> loop), <code>main.js</code> (entry, ad/analytics/session bootstrap), <code>monetization/index.js</code> (MonetizationBus, strategy engine), <code>bot/trainer.js</code> (browser RL), <code>bot/rlPanel.js</code> (training UI)
Layer 3: Domain Services



3.3 Cross-Platform Architecture

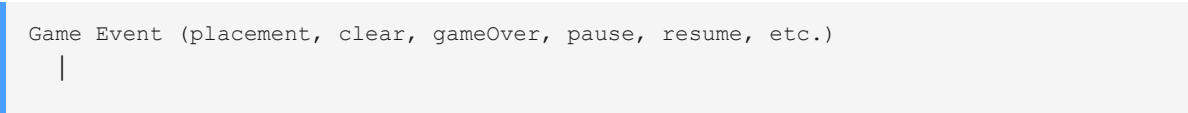
OpenBlock targets four platforms from a single codebase, with shared game logic and configuration:

Platform	Technology	Feature Set	Rendering
Web (primary)	Vite + vanilla JS	Full features	Canvas 2D / WebGL
WeChat Mini Program	Adapted build, wx APIs	Excludes RL, monitoring; subset of monetization	Canvas 2D
iOS/Android	Capacitor WebView	Full features via embedded WebView	Full
Cocos Creator	Cocos 3.x, separate renderer	In development; native performance target	Cocos native

Cross-platform contract. All platforms share `shared/game_rules.json` (configuration), `shared/shapes.json` (shape definitions), and the same feature encoding (`rl_pytorch/features.py` ↔ `web/src/bot/features.js`). A cross-language test suite guarantees equivalence: `tests/test_spawn_step_difficulty.py` and `tests/spawnStepDifficulty.test.js` share fixture `tests/fixtures/spawnStepDifficulty.cases.json` and assert bitwise-identical outputs for the same inputs.

3.4 Data Flow and Event Bus

A unified event bus connects game events to all consuming subsystems:




```

└─ MonetizationBus.emit(event) — ①
  └─ MonetizationDecisionEngine.evaluate() → ad/show, iap/offer
    └─ MonetizationLogger.record(decision, outcome)

└─ PlayerProfile.update(event) — ②
  └─ _computeRawSkill() → skillLevel (EMA)
  └─ _updateFlowDeviation() → flowState
  └─ _updateFrustration() → frustrationLevel
  └─ _updateMomentum() → momentum
  └─ _updateLifecycle() → lifecycleStage

└─ SessionEval.record(event) — ③
  └─ evaluatePlacement() → placementQuality[5]
  └─ evaluateRound() → roundQuality, regret components
  └─ sessionEvalRecord.update()

└─ SpawnContext.update(event) — ④
  └─ _updateHistory() → recent shape/dock trajectory
  └─ _recordFeedback() → feedbackBias update
  └─ _updateDiagnostics() → spawn decision trace

└─ state_history INSERT — ⑤
  └─ (session_id, step, grid_blob, dock_ids, action, clear_count, score,
    player_snapshot)

```

This event-driven design enables the monetization framework to observe game state without modifying it (Principle 1: minimal intrusion), allows new consumers to be added without changing the game engine, and provides a complete audit trail for every game event across all subsystems.

3.5 Configuration-Driven Design

`shared/game_rules.json` serves as the single source of truth (SSOT) for all algorithm parameters. The file is organized into semantic sections, each consumed by specific modules:

Section	Key Parameters	Consumers
featureEncoding	stateScalarDim=65, maxGridWidth=8, dockSlots=3, actionNorm	features.js, features.py, model.py
adaptiveSpawn	10-tier profileLevels, constructiveSpawn, difficultyBucket, stress weights	adaptiveSpawn.js, blockSpawn.js
rlRewardShaping	Reward weights, auxiliary supervision coefficients, outcome value mix, topology aux dim	simulator.py, train.py, model.py
playerAbilityModel	Skill weights, EMA decay rates, flow thresholds, frustration levels	playerProfile.js, abilityModel.js
clearScoring	single_line=20, combo activationCount=3, icon bonusLineMult=5, perfectClearMult=10	clearScoring.js, simulator.py
rlCurriculum	difficultyBucket.enabled, stages with untilEpisode/maxScd, retryCap=6	simulator.py, train.py
conditionToken	Arc (5) and intent (6) one-hot vocabulary, samplingProb=0.6	features.js, features.py, train.py

A project-wide “**no magic numbers**” principle is enforced: any numerical constant appearing in algorithm code must be read from `game_rules.json` or a database table. Code that hardcodes a value is rejected at code

review. This ensures that all tuning, A/B experimentation, and difficulty calibration can be performed by modifying a JSON file—without touching code and without requiring a redeployment.

4. Real-Time Player Profiling

4.1 Design Constraints

The player profiling system operates under three binding constraints:

1. **Browser-only execution:** All computations must run in the browser with sub-millisecond overhead. No server round-trip, no neural inference engine, no WebAssembly dependency for the profiling core.
2. **Stability from sparse data:** The system must produce stable estimates from noisy, sparse observations. A player may have only 5 placement events in their first game—the system must produce a reasonable skill estimate without overfitting.
3. **Interpretability:** Both developers debugging spawn decisions and players viewing their own ability metrics must be able to understand what each number means and where it came from.

4.2 Skill Estimation

Instantaneous raw skill is computed at each step as a weighted linear combination of five behavioral dimensions, each normalized to $[0, 1]$:

$$r_t^{skill} = 0.15 \cdot \tau_{think} + 0.30 \cdot \tau_{clear} + 0.20 \cdot \tau_{combo} + 0.20 \cdot \tau_{miss} + 0.15 \cdot \tau_{load}$$

where:

Component	Definition	Interpretation
τ_{think}	$1 - \min(1, thinkMs/2000)$	Decision speed: faster = higher skill
τ_{clear}	$\min(1, clears/3)$	Clear efficiency per placement
τ_{combo}	$\min(1, comboCount/5)$	Combo chain maintenance
τ_{miss}	$1 - wastedPlacementsRatio$	Placement efficiency (fewer “dead” placements)
τ_{load}	$1 - cognitiveLoadIndex$	Lower cognitive load = higher skill

This raw score is smoothed via exponentially-weighted moving average (EMA) with adaptive decay:

$$s_t^{skill} = s_{t-1}^{skill} + \alpha \cdot (r_t^{skill} - s_{t-1}^{skill})$$

$$\alpha = \begin{cases} 0.35 & \text{if } stepsThisGame \leq 5 \\ 0.15 & \text{otherwise} \end{cases}$$

The dual-rate design provides rapid adaptation for new and returning players (first 5 steps of each game) while maintaining stable tracking for experienced players in sustained play. The 0.15 steady-state rate means the EMA half-life is approximately $\ln(2)/0.15 \approx 4.6$ steps—adaptation is responsive but not jittery.

Historical fusion. To capture longer-term skill trends while remaining responsive to recent performance, a historical fusion layer blends the current smoothed estimate with an exponentially-weighted historical average:

$$histSkill = \frac{\sum_{i=1}^{n-1} 0.85^{n-1-i} \cdot skill_i}{\sum_{i=1}^{n-1} 0.85^{n-1-i}}$$

$$skillLevel = (1 - w_{hist}) \cdot s_t^{skill} + w_{hist} \cdot histSkill$$

where $w_{hist} = (1 - w_{smooth}) \cdot confidence$. The confidence term reflects the reliability of the current session estimate: early sessions with few observations have low confidence and rely more on historical data; mature sessions with consistent behavior have high confidence and override stale history.

4.3 Flow Detection

Flow state—the optimal experience zone where challenge approximately matches skill—is formalized after Csikszentmihalyi’s flow theory:

$$F(t) = \left| \frac{boardPressure(B_t)}{\max(0.05, skillLevel)} - 1 \right|$$

The 0.05 floor prevents division by zero for brand-new players. Board pressure is a composite of three factors:

$$boardPressure = 0.45 \cdot fillRatio + 0.35 \cdot clearDeficit + 0.20 \cdot cogLoad$$

Component	Channel Weight	Definition
fillRatio	0.45	Occupancy ratio (<i>filled</i> /64)
clearDeficit	0.35	$1 - clearRate$, where $clearRate = clears/placements$
cogLoad	0.20	Normalized cognitive load from board topology complexity

Flow state classification is performed by a rule tree:

$$flowState = \begin{cases} bored & \text{if } F(t) < 0.9 \\ flow & \text{if } 0.9 \leq F(t) \leq 1.3 \\ anxious & \text{if } F(t) > 1.3 \end{cases}$$

The asymmetry in the thresholds (−0.1 below vs +0.3 above) reflects an intentional design bias: it is safer to slightly under-challenge a player (boredom can be fixed with a harder dock) than to over-challenge them (anxiety leads to churn). The wider flow band on the upper side provides a buffer before the system classifies a player as anxious and triggers relief mechanisms.

4.4 Frustration and Distress Tracking

Frustration is tracked via consecutive no-clear steps:

$$frustrationLevel_t = \begin{cases} frustrationLevel_{t-1} + 1 & \text{if } no\ lines\ cleared\ at\ step\ t \\ 0 & \text{if } lines\ cleared\ at\ step\ t \end{cases}$$

Escalating thresholds trigger progressively stronger interventions:

Level	Threshold	Intervention	Mechanism
Warning	3	Spawn engine injects easier shape	<code>clearGuarantee +0.5</code> , <code>difficultyTarget -0.1</code>
IAP hint	4	Monetization system offers rescue item	<code>iap/rescue_offer</code> trigger
Rescue	5	Mandatory relief: spawn engine override	<code>clearGuarantee +2</code> , <code>warmMode = true</code>

Distress signal. While frustration captures short-term “stuckness,” distress captures cumulative structural damage—the long-term degradation of board quality:

$$distress = 0.4 \cdot holesRatio + 0.3 \cdot transitionsRatio + 0.2 \cdot wellsRatio + 0.1 \cdot concaveRatio$$

Each component is normalized by its maximum expected value on an 8×8 grid. The `distress` signal modulates the `feedbackBias` damping mechanism (§6.6): when a player is clearing lines (positive

`feedbackBias`) but accumulating structural damage (elevated `distress`), the system reduces the positive bias to prevent difficulty escalation on a structurally compromised board.

4.5 Momentum and Streak Signals

Momentum captures the direction and rate of performance change using a sliding window comparison:

$$\Delta = \frac{clears_{recentWindow}}{placements_{recentWindow}} - \frac{clears_{baselineWindow}}{placements_{baselineWindow}}$$

$$momentum = clamp\left(\frac{\Delta}{0.3}, -1, 1\right)$$

The window sizes are configurable (defaults: recent = 8 placements, baseline = 24 placements). The 0.3 normalization constant maps a typical performance swing (30% change in clear rate) to momentum = ±1.

Run streak. A between-game streak signal tracks the player’s trajectory across multiple games:

$$runStreak_g = runStreak_{g-1} + sign(score_g/PB - streakThreshold)$$

where `streakThreshold` is a configurable ratio (default: 0.6). A positive streak (sustained above-threshold performance) signals the spawn engine to gradually increase difficulty; a negative streak triggers the Warm Run protection system.

4.6 Lifecycle × Maturity Matrix (25-Grid)

OpenBlock models each player along two orthogonal axes, creating a 5×5 differentiation matrix:

	M4 (Expert ≥90th)	M3 (Skilled ≥80th)	M2 (Intermediate ≥60th)	M1 (Novice ≥40th)	M0 (Beginner <40th)
S0 (New)	—	—	Accelerated ramp	Extended tutorial	Maximum protection
S1 (Active)	Full challenge	Graduated challenge	Standard progression	Protected difficulty	Guided progression
S2 (Plateau)	Maintain engagement	Plateau breaker	Fresh content push	Overwhelm guard	Frustration shield
S3 (Churn)	Re-engagement reward	Churn prevention	Retention offer	Comeback incentive	Rescue package
S4 (Return)	Welcome-back bonus	Gentle re-onboarding	Tutorial refresher	Full re-tutorial	Complete restart

Lifecycle stage derivation:

$$lifecycle = f(daysSinceInstall, totalSessions, daysSinceLastActive)$$

This is a three-input AND gate: all three signals must agree on the stage classification. If signals conflict (e.g., `daysSinceInstall` suggests S0 but `totalSessions` suggests S2), the system defaults to the more conservative (more protective) classification.

Maturity band derivation:

$$maturity = g(skillLevel, historicalSkillDistribution)$$

Skill scores are compared against the global player distribution, with thresholds at the 90th, 80th, 60th, and 40th percentiles. The maturity band is updated once per session (not per step) to prevent intra-game band flickering.

4.7 Offline Aggregate Profile (Player Analytics)

Complementing the real-time browser profile, an offline aggregate analytics pipeline (`playerAnalytics`) consumes frame-level time series data from `move_sequences.frames[].ps` to produce:

Six-dimensional ability vector with confidence bounds:

Dimension	Definition	Confidence Metric
topology	Board structure management (avoiding holes, maintaining mobility)	Within-session variance
scoring	Clear efficiency per placement	Session count
execution	Placement speed and accuracy	Observation count
reaction	Response to difficulty changes	Difficulty variation experienced
survival	Longevity under adverse conditions	Distress exposure
consistency	Performance stability across sessions	Cross-session variance

Temporal traits:

- **Trend:** slope of the linear regression of skill over the last 10 sessions.
- **Endurance:** ratio of late-session performance to early-session performance.
- **Clutch:** performance differential when score > 80% PB vs. baseline.

Spawn advice layer:

- `shapeCompetence`: per-shape-category clear rates, used to bias the spawn weight chain.
- `comfortFillBand`: the fill ratio range where the player performs best.
- `topologyForm.weakness`: board topology patterns (e.g., many wells, high transitions) associated with poor outcomes.
- Personalized `relief` and `delight` thresholds: at what frustration level does this specific player respond to intervention?

This offline profile complements the real-time system: the real-time profile drives per-frame spawn decisions, while the offline profile provides session-level and cross-session context for personalization, cohort analysis, and cold-start priors.

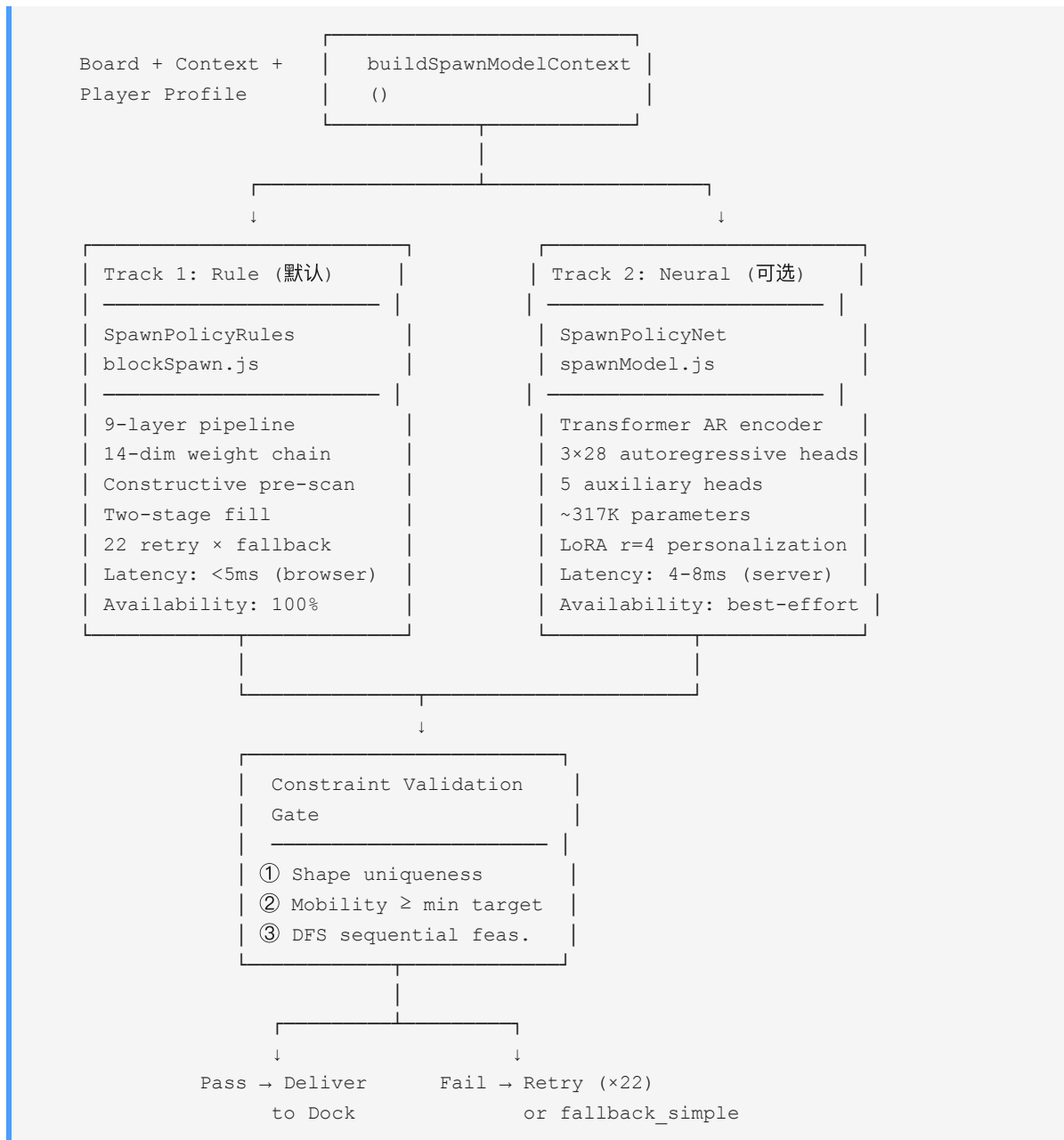
5. The Spawn Engine: Programmatic Content Generation

5.1 Design Space and Constraints

The spawn engine must select 3 polyomino pieces from a catalog of 28 shapes given an 8×8 board with up to 64 occupied cells. The raw combinatorial space is $28^3 = 21,952$ possible triplets, but this is dwarfed by the state-dependent constraints: each shape only has legal placements in a subset of board positions, and the three shapes must be jointly sequentially feasible. The engine must balance four competing objectives:

1. **Solvability:** every delivered dock must be fully placeable (hard constraint, verified by DFS).
2. **Engagement:** pieces should create clear opportunities to sustain the core satisfaction loop.
3. **Challenge calibration:** difficulty should track the player’s current skill-flow state.
4. **Delight:** occasional high-reward combinations (multi-clears, icon bonuses, perfect clears) should feel earned, not random.

5.2 Dual-Track Architecture



Design rationale for dual-track:

- Track 1 (rule) is the **always-available safety net**. It runs entirely in the browser with sub-5ms latency, is fully deterministic and explainable, and has zero external dependencies. It is the default path and the guaranteed fallback.
- Track 2 (neural) is the **asymmetric upside**. It learns distributions from data that the rule engine’s hand-tuned weights cannot express. Its output is validated by the same constraint gate as Track 1; any failure triggers automatic, transparent fallback. This means the neural track can never produce *worse* output than the rule track (worst case: fallback), but can produce *better* output when it has learned useful patterns.

Both tracks consume the same context object (`buildSpawnModelContext()`), which includes: difficulty mode, ability vector (6-dim), player profile snapshot (skill, flow, frustration, momentum), board topology, in-game rhythm signals, between-game arc parameters, recent spawn history, and rule-track spawn hints.

5.3 The Nine-Layer Generation Pipeline

The rule-track engine (`generateDockShapes` in `blockSpawn.js`) decomposes the monolithic spawn problem into nine sequentially composed layers. Each layer addresses a specific, well-defined sub-problem, reads from a specific section of `game_rules.json`, and produces structured output consumed by the next layer:

Layer 1: Input Assembly

Inputs consumed: Board state `B_t`, player profile `π_t`, spawn context `ctx_t` (recent history, difficulty targets, arc phase), spawn intent `I_t`.

Processing: Aggregates heterogeneous inputs into a unified `SpawnModelContext` object. This includes normalizing all inputs to consistent scales (e.g., `difficultyTarget` $\in [0,1]$, `fillRatio` $\in [0,1]$) and computing derived signals (e.g., `crowding` from `fill` + `contiguous_regions` + `transitions`).

Output: `ctx` object with all inputs validated and normalized.

Layer 2: Board Perception

Analysis functions called:

- `analyzeBoardTopology(grid)` : returns fill ratio, row/column extremes, hole count (unfillable empty cells), row/col transitions (0↔1 boundary counts), well depth sum, close-to-full line counts (1 cell away, 2 cells away), mobility (total legal positions across all shapes), contiguous empty regions (4-connected components), concave corners count. All features are computed in $O(n^2)$ where $n = 8$ for the standard grid.
- `spatialPlanningFeatures(grid)` : returns `regionEntropy` (diversity of empty region sizes), `largestRegionRatio` (largest connected empty region / total empty cells), `smallRegionCellRatio` (small isolated empty regions / total empty cells). These three features quantify the *fragmentation* of the remaining empty space—a critical indicator of whether long-bar shapes can still fit.
- `colorSummary(grid, dock)` : returns per-color occupancy ratios and single-color-line potentials (rows/columns where all occupied cells share the same color).

Output: Topology vector (holes, transitions, wells, `close1`, `close2`, mobility, `fill`, `contiguous_regions`, `concave_corners`), spatial planning vector (3-dim), color summary vector (19-dim).

Layer 3: Score Construction

Fusion of 17 signals into a composite spawn step difficulty (SCD) score:

$$SCD = \frac{\sum_{j=1}^{17} w_j \cdot s_j}{\sum w_j}$$

The 17 signals are grouped into six categories, each with configurable weights:

Category	Weight	Signals
scd (density)	0.26	Total block cells / free cells, normalized by saturation point
board (topology)	0.18	Board difficulty from topology features
flexibility	0.18	Mobility inverse ($1 - \text{placements}/\text{max}$), minimum flexibility across dock blocks
solution	0.13	DFS solution count (or 0 if truncated at budget)
killer	0.13	$(\text{killerCount} + \text{longBarCount} \times 0.5) / 3$, where killer shapes are large (≥ 5 cells) or long-bars with low placement counts
fragmentation	0.12	$\text{regionEntropy} \times 0.6 + \text{smallRegionCellRatio} \times 0.4$

Bucket classification:

$$\text{bucket}(SCD) = \begin{cases} \text{trivial} & SCD \leq 0.2 \\ \text{easy} & 0.2 < SCD \leq 0.4 \\ \text{standard} & 0.4 < SCD \leq 0.6 \\ \text{hard} & 0.6 < SCD \leq 0.8 \\ \text{extreme} & SCD > 0.8 \end{cases}$$

Layer 4: Priority Selection

The 10-intent priority scheduler maps the current `spawnIntent` to concrete shape selection parameters:

Priority	Intent	clearGuarantee	Shape Pool Bias	Size Preference
115	warm_run	2-3	Easy blocks (squares, small lines), flush-clears	Small preferred
102	pb_chase	1	Slightly harder, PB-relevant (large clears)	Large preferred
100	relief	2	Stress-relieving (easy to place, high mobility)	Small preferred
95	delight_starved	1	Multi-clear opportunities, icon-bonus chains	Mixed
90	engage	1	Balanced with high variety	Mixed
80	harvest	0-1	Efficiency-optimized, combo-sustaining	Medium preferred
70	pressure	0	Graduated challenge, deliberate difficulty	Large allowed
60	sprint	0	Speed-oriented, simple shapes	Small preferred
50	flow	0	Standard weighted distribution	Neutral
0	maintain	0	Pure random weighted, no bias	Neutral

The `clearGuarantee` parameter determines how many dock slots (out of 3) are pre-allocated for shapes that can immediately clear at least one line. A value of 2 means the engine will try to fill 2 of the 3 dock slots with clearing shapes before filling the remainder via weighted sampling.

Layer 5: Weighted Completion

A 14-dimensional weight chain drives two-stage constructive filling. Each weight amplifies or attenuates the sampling probability of shapes that serve the corresponding objective:

Shape base weights (12 dimensions):

Weight	Range	Effect of Higher Value
gapFills	[0, 2]	Prefer shapes that can fill 1-cell gaps in near-complete lines
multiClear	[0, 3]	Prefer shapes likely to clear ≥ 2 lines simultaneously
holeReduce	[0, 2]	Prefer shapes whose placement reduces unfillable hole count
mobility	[0, 2]	Prefer shapes that leave high post-placement mobility
salvage	[0, 1.5]	Prefer shapes viable when total mobility is critically low
pcPotential	[0, 2]	Prefer shapes that increase perfect-clear probability
clearGuarantee	[0, 2]	Directly amplify clearing shapes when guarantee quota is active
comboFwd	[0, 1.5]	Prefer shapes that sustain or extend combo chains
diversity	[0, 1.5]	Penalize shapes from over-represented categories

Weight	Range	Effect of Higher Value
novelty	[0, 1]	Penalize shapes recently seen in the player's history
stressBalance	[0, 2]	Modulate weights based on stress level
monoTarget	[0, 2]	Prefer shapes that match the dominant color on near-complete lines

Enhancement layer (2 dimensions):

Weight	Effect
delightBoost	Amplified when <code>isDelightStarved = true</code> (player hasn't experienced a satisfying multi-clear or icon bonus recently). Multiplies the <code>multiClear</code> , <code>pcPotential</code> , and <code>gapFills</code> weights by up to 1.5×.
stressOverride	Clamps the maximum SCD of generated triplets when <code>stress</code> exceeds configurable thresholds. Overrides <code>clearGuarantee</code> quota if necessary.

Two-stage constructive fill:

```
function weighted_completion(ctx, intent, clearGuarantee, shapeWeights):
    dock ← [null, null, null]

    # Stage 1: Clear seats (guaranteed clearing shapes)
    clear_seats ← min(clearGuarantee, 3)
    clearing_shapes ← select_shapes_with_clear_potential(board, ctx)
    for i in 1..clear_seats:
        if clearing_shapes not empty:
            dock[i] ← weighted_sample(clearing_shapes, shapeWeights)

    # Stage 2: Weighted fill (remaining slots)
    remaining_slots ← indices where dock is null
    augmented_pool ← all_shapes × enhance_weights(shapeWeights)
    for slot in remaining_slots:
        candidate ← weighted_sample(augmented_pool, shapeWeights)
        if candidate != any(dock[0..slot-1]): # uniqueness check
            dock[slot] ← candidate
        else:
            retry with different random seed

    return dock
```

Constructive pre-scan (C1/C2/C3) runs during weighted completion. Before sampling, the engine examines the current board:

- **C1 (Completer):** Scans for rows or columns exactly one cell from completion. If found, amplifies the weight of shapes that can fill the missing cell. The amplification factor is proportional to the number of completable lines the shape would finish.
- **C2 (Setup):** Scans for configurations where placing a specific shape creates a *future* near-complete line—even if the placement itself clears nothing. This is effectively one-step lookahead: “if I place shape X here, it won’t clear now, but it will create a situation where shape Y could clear next turn.” C2 operates on a limited search budget (~50 shape × position evaluations).
- **C3 (Order Anchor):** When multiple constructive opportunities exist, C3 determines the optimal ordering for the three dock slots. It evaluates each ordering by: (a) expected total clears across the three placements, and (b) post-dock board topology quality (holes, mobility). The ordering with the best composite score becomes the final triplet.

Between stages, a **PEOG clamp** (Placement Efficiency and Operator Guard) restricts the constructive operator candidate set. PEOG prevents the engine from selecting overly complex constructions that require a specific sequence of placements to work—constructions that might be theoretically optimal but practically frustrating for a human player. The clamp limits the maximum number of constructive operators to 2 (out of 3 possible), ensuring at least one slot is always a “natural” (unconstructed) choice.

Layer 6: Constraint Verification

Three sequential gates, any failure triggers retry:

Gate 1: Shape uniqueness.

$$s_1 \neq s_2 \neq s_3$$

Trivial O(1) check. Violation: duplicates in dock.

Gate 2: Mobility check.

$$M(B_t, \{s_1, s_2, s_3\}) = \sum_{k=1}^3 |\text{legalPositions}(B_t, s_k)| \geq M_{\min}$$

where $M_{\min}$ is a function of fill ratio: $M_{\min} = 10$ at low fill, linearly decreasing to $M_{\min} = 3$ at fill ≥ 0.75 . This prevents the system from delivering a dock where the player has only 1–2 total legal moves, which feels unfair.

Gate 3: DFS sequential feasibility. As described in §2.4. Budget: 200 nodes, leaf cap: 1 (binary pass/fail). This is the most expensive check (~50–200ms when the board is difficult) and the ultimate guarantee of playability.

Layer 7: Injection Optimization

Special event blocks are injected when conditions align. These overwrite regular weighted selections only after passing the same constraint gates:

- **Flush-clear injection:** When the player is `delight_starved` and the board has a near-complete line that can be filled by a specific shape, that shape is injected into the dock (probability modulated by `delightBoost`).
- **Icon-bonus injection:** When a row/column has ≥ 6 cells of the same color and is 1–2 cells from completion, a shape matching the dominant color with a compatible footprint is injected.
- **Perfect-clear injection:** When the board fill is $\leq 50\%$ and the current pieces could theoretically clear the entire board in 3 placements (estimated by heuristic), the engine amplifies `pcPotential` weights and may inject a shape that maximizes clear probability.

Injections are rate-limited: a `constructiveRetry` counter tracks consecutive failed injections (where the injected shape was provided but the player couldn’t capitalize), and after 2 consecutive failures, injection probability is halved for the next 3 rounds.

Layer 8: Output Delivery

The final triplet is confirmed. Diagnostic metadata is attached to the spawn context:

- `spawnIntent` and priority level
- `spawnDifficulty` (SCD) and its bucket
- Which slots were filled by `clearSeats` vs. `weightedFill`
- Which constructive operators fired (C1/C2/C3)
- Constraint check outcomes (all three gates, with failure reasons if any)
- The full 14-dimensional weight vector used for this spawn

Layer 9: Color Display

Color assignment is decoupled from shape selection (by design). The function `monoNearFullLineColorWeights(grid)` scans rows and columns that are 1–2 cells from completion, identifies the dominant color among existing cells, and amplifies the sampling weight for shapes of that color. This creates icon-bonus opportunities without compromising the shape selection: the shape chosen by layers 1–8 determines the geometry; layer 9 determines the surface color.

Color weights are attenuated by a configurable `iconBonusTarget` parameter (default: 0.3), which controls how aggressively the system biases toward icon-matching. Higher values create more icon bonuses but may feel manipulative; lower values create a more natural distribution.

5.4 Between-Game Difficulty (RoR — Rate of Return)

Between individual games, difficulty progression is modulated by the Rate of Return (RoR) system. The player's current arc stage determines the baseline difficulty trajectory:

Arc stage derivation:

Arc	Derivation	Difficulty Modulation
Opener	First 1–2 games of session	$d^*_{base} \times 0.7$ (warm-up)
Momentum	Games 3–8, performance improving	$d^*_{base} \times 1.0$ (standard)
Peak	Games 9–15, best performance	$d^*_{base} \times 1.15$ (challenge)
Fatigue	Games 16+, performance declining	$d^*_{base} \times 0.85$ (wind-down)
Cooldown	Session ended; next session starts	Reset to Opener

Humped difficulty curve within a session:

$$d^*(n) = d_{base} \cdot \left(1 + h \cdot \frac{n}{N} \cdot \left(1 - \frac{n}{N}\right)\right)$$

where n is the current game number in the session, N is the expected session length (estimated from historical data), and h is the hump height (configurable per arc stage: 0.15 for momentum, 0.25 for peak, 0.10 for fatigue).

5×5×5 cubic modulation matrix. Fine-grained difficulty targeting uses a three-dimensional lookup indexed by arc stage (5 levels), session offset (5 quantized positions within the arc), and PB ratio (5 buckets: <0.3, 0.3–0.6, 0.6–0.8, 0.8–0.95, ≥0.95 of PB). This produces 125 distinct difficulty profiles, each with its own `d*_base` and shape weight adjustments.

5.5 Spawn Step Difficulty Metrics

At each spawn decision, the SCD computation produces a 4-dimensional feature vector (extended to 12 dimensions for RL auxiliary supervision):

Original 4 dimensions (used in state features and spawn tuning):

Index	Name	Formula	Range
0	<code>scdNorm</code>	$\text{clamp}_{[0,1]}(\text{scd}/\text{scdSaturation})$, where $\text{scd} = \sum \text{cells}(s_k)/(\text{freeCells} + \epsilon)$	[0,1]
1	<code>comboCellsNorm</code>	$\text{clamp}_{[0,1]}(\sum \text{cells}(s_k)/\text{comboCellsNorm})$, default norm = 15	[0,1]
2	<code>comboKillerNorm</code>	$\text{clamp}_{[0,1]}(\text{killerCount}/\text{dockSlots})$	[0,1]
3	<code>comboLongBarNorm</code>	$\text{clamp}_{[0,1]}(\text{longBarCount}/\text{dockSlots})$	[0,1]

v13 extension: 8 per-shape placeability dimensions:

Index	Shape	Normalization Denominator	Theoretical Max Positions
4	1×4	40	$(8-1+1) \times (8-4+1) = 8 \times 5$
5	4×1	40	$(8-4+1) \times (8-1+1) = 5 \times 8$
6	1×5	32	$(8-1+1) \times (8-5+1) = 8 \times 4$
7	5×1	32	$(8-5+1) \times (8-1+1) = 4 \times 8$
8	2×2	49	$(8-2+1) \times (8-2+1) = 7 \times 7$
9	3×3	36	$(8-3+1) \times (8-3+1) = 6 \times 6$
10	T-up	42	$(8-2+1) \times (8-3+1) = 7 \times 6$
11	L3-a	49	$(8-2+1) \times (8-2+1) = 7 \times 7$

Each placeability dimension is computed as $\text{clamp}_{[0,1]}(\text{len}(\text{getLegalPositions}(B_t, \text{shape}))/\text{norm})$. The eight fixed shapes cover the four long-bar pieces (the primary bottleneck), two square pieces (baseline), and two complex pieces (T and L, the most commonly appearing non-line shapes). Computational cost: ~0.16ms with Numba JIT (8 calls to the vectorized legal position kernel).

5.6 Guard Rails and Fallback

Multiple layers of protection guarantee that every delivered dock is playable:

1. **22 retry attempts** ($\text{'MAX}_{\text{S}}\text{PAWN}_{\text{A}}\text{TTEMPTS}' = 22$): Each retry re-executes the full two-stage construction with a different random seed. The high retry count is feasible because the constraint gate (specifically the DFS check) is the expensive step; the weighted construction is fast (~0.5ms). In practice, over 99.9% of docks pass within 3 attempts; the 22-retry budget is a safety margin for edge-case board states.
2. **fallback_simple**: If all 22 retries fail, a simplified path uniformly randomly samples shapes from the full catalog until a feasible triplet is found. This path has no difficulty targeting or constructive optimization—it is a pure safety net that guarantees a playable dock.
3. **Warm Run clamping** (`applyWarmRun`): For new (S0), returning (S4), and distressed players, a post-hoc override adjusts the shape weights: `easyWeights` (squares, short lines, small rects) are amplified by 1.5–2.0×, and `hardWeights` (long bars, large rects, J-shapes) are attenuated by 0.3–0.5×. The warm budget decays over the session: $\text{warmBudget}_g = \text{warmBudget}_{\{g-1\}} \times 0.85 - \text{warmCost}_g$, terminating when budget reaches zero.
4. **Overload protection**: When fill ratio exceeds 0.70, the difficulty target `d*` is automatically reduced by up to 0.20, proportionally to $(\text{fill} - 0.70)/0.30$. This addresses the long-bar bottleneck directly: at fill ≥ 0.70 , the system recognizes that long bars are becoming unplaceable and reduces difficulty to avoid generating impossible triplets.

6. Scoring, Placement Quality, and Evaluation

6.1 Clear Scoring Formula

OpenBlock’s scoring system uses a quadratic formula that strongly rewards multiple simultaneous clears:

Base score:

$$\text{score}_{\text{base}} = 20 \cdot c^2$$

where $c = \text{rowsCleared} + \text{columnsCleared}$, with $0 \leq c \leq 6$ (maximum: 3 rows + 3 columns on an 8×8 grid). The quadratic scaling creates strong non-linearity: a single-line clear ($c=1$) earns 20 points, while a triple-

line clear ($c=3$) earns 180 points— $9\times$ the reward for $3\times$ the effort.

Icon bonus. If any cleared row or column consists entirely of blocks sharing the same icon (color), those lines earn a multiplier:

$$iconBonus = 10 \cdot c \cdot (iconBonusLineMult - 1) \cdot b$$

where b is the number of icon-matched lines and $iconBonusLineMult = 5$ by default. This rewards strategic placement toward color homogeneity.

Perfect clear bonus. If every cell on the board becomes empty after clearing:

$$perfectMult = 10$$

This high multiplier ($10\times$) creates a compelling risk-reward dynamic: attempting a perfect clear risks wasting placements on suboptimal positions, but succeeding yields an outsized score boost.

Combo multiplier. Consecutive clear placements increment a combo counter, with a grace window of 3 non-clearing placements before the combo resets:

$$m_{combo} = \min(m_{max}, 1 + \max(0, comboCount - activationCount + 1) \cdot stepBonus)$$

with default parameters: $activationCount = 3$, $stepBonus = 0.0$, $maxMultiplier = 1.0$. These neutral defaults mean combo multiplier is effectively disabled but can be activated via configuration to reward sustained clear streaks.

Full score formula:

$$score_{placement} = (20c^2 + c \cdot 40 \cdot b) \cdot \begin{cases} 10 & \text{if } perfectClear \\ 1 & \text{otherwise} \end{cases} \cdot m_{combo}$$

6.2 Placement Quality Evaluation

Each placement is evaluated against the theoretical optimum for that board-dock configuration. The evaluation produces a 5-dimensional quality vector:

Dimension	Computation	Range
Topology delta	$\Phi(B_{after}) - \Phi(B_{before})$, where Φ is the board potential function (§7.1)	$[-1, 1]$
Mobility delta	$\frac{M_{after} - M_{before}}{\max(M_{before}, 1)}$	$[-1, 1]$
Clear potential	$\min(1, c/3)$	$[0, 1]$
Near-full proximity	$\frac{nearFullAfter}{nearFullBefore+1}$	$[0, 1]$
Salvage quality	$\begin{cases} \min(1, c/3) & M_{before} \leq 4 \\ 0 & \text{otherwise} \end{cases}$	$[0, 1]$

Regret computation. For each placement, the evaluator computes the regret—the gap between the optimal placement’s quality and the chosen placement’s quality:

$$regret = \min_{optimal a^* \in legal} \|Q(a^*) - Q(a_{chosen})\|$$

where $Q(a)$ is the composite quality score. Regret is normalized by a configurable denominator ($regretNorm = 8.0$) and clamped to $[0, 1]$.

Special classifications:

- **forced_bad**: $holesAfter - holesBefore \geq 2$. The dock was structurally adverse—even optimal play couldn’t prevent board degradation. High **forced_bad** rate ($>15\%$ of rounds) triggers spawn engine relief.
- **salvage**: $M_{before} \leq 4 \wedge c \geq 2$. The player achieved a multi-clear despite critically low mobility—skillful play under constraint. High salvage rate indicates the player is performing above the

system's expectation of their ability.

6.3 Round Quality

After a full dock (3 placements or game-over), the round receives a quality classification with three regret components:

1. **Order regret:** Was the placement order optimal? $orderRegret = Q(optimalOrdering) - Q(chosenOrdering)$.
2. **Path regret:** Was each individual placement optimal given the chosen order? $pathRegret = \frac{1}{3} \sum_{i=1}^3 regret(step_i)$.
3. **Payoff regret:** Did the round achieve the expected clear reward? $payoffRegret = \max(0, \mathbb{E}[c] - c_{actual})$.

These components aggregate into a `roundQuality` score stored in `sessionEvalRecord`.

6.4 The Feedback Closed Loop

A real-time feedback bias signal closes the loop between player action and spawn difficulty, creating a sub-second response latency:

```
player clears MORE lines than expected → feedbackBias += 0.02
player clears FEWER lines than expected → feedbackBias -= 0.02
```

The bias is clamped to $[-0.15, 0.15]$ and feeds directly into the stress computation:

$$stress = \sum_j w_j \cdot s_j + feedbackBias$$

Distress damping. To prevent the system from increasing difficulty on a player who clears lines but destroys their board structure:

$$feedbackBias_{effective} = feedbackBias - \min(0.08, feedbackBias \cdot 0.5 \cdot distress)$$

This only applies when $feedbackBias > 0$ (the system thinks the player can handle more) AND $distress > 0$ (but their board is deteriorating). The damping is proportional to both the bias magnitude and the distress level.

Four-layer evaluation → adaptiveSpawn feedback:

Layer	Signal	Spawn Response
Step (placementQuality)	<code>consecutiveForcedBad ≥ 2</code>	<code>clearGuarantee += 2</code>
Round (roundQuality)	<code>lastRoundClassification = forced_bad</code>	<code>targetSolutionRange.max += 2</code>
Session (sessionEval)	<code>qualityTrend = declining</code>	Reduce <code>d*_base</code> by 10%
Between-game audit (RoR)	<code>avgQuality < baseline × 0.85</code>	Trigger <code>warmRunActive</code> for next game

7. Models and Algorithms: Complete Architecture

OpenBlock's AI system comprises **six distinct models** operating at three architectural layers, each with precisely defined responsibilities and interfaces:

```

L1 – Spawn Layer (What blocks to show?)
├─ SpawnPolicyRules  (Rule engine, always-on, zero parameters)
└─ SpawnPolicyNet    (Transformer neural model, ~317K params, optional)

L2 – Parameter Layer (What  $\theta$  for L1?)
└─ SpawnParamTuner   (ResNet-MLP, ~325K params, offline training)

L3 – Placement Layer (Where to put a given block?)
├─ ConvSharedPolicyValueNet (PyTorch RL, ~188K params, server training)
├─ LightPolicyValueNet      (Lightweight MLP, ~28K params, CPU)
├─ LightSharedPolicyValueNet (CNN-light shared, ~50K params, CPU)
├─ Browser LinearAgent      (Browser REINFORCE, ~28K params, client-side)
└─ MLX RL                   (Apple Silicon RL, ~28-50K params, Metal-accelerated)

```

The models are organized along two orthogonal design axes:

- **L1/L2/L3 separation:** Spawn (L1), parameter tuning (L2), and placement (L3) are independent problems solved by independent models, connected only through shared feature encoding (`game_rules.json`).
- **Rule vs. Learned separation:** Each layer has a rule-based path (deterministic, always available, explainable) and optional learned paths (data-driven, potentially superior, automatically failsafe through constraint validation gates).

7.1 Model Overview and Comparison

Model	Layer	Algorithm	Parameters	Training Data	Latency	Deployment
SpawnPolicyRules	L1 (Spawn)	Multi-signal heuristic + weighted sampling + constructive pre-scan + DFS gate	0 (rule-based)	None	<5 ms	Browser (always)
SpawnPolicyNet	L1 (Spawn)	Transformer encoder + autoregressive slot decoder + multi-task supervision + LoRA	~317K	Player replays + rule-engine synthetic + self-play	4-8 ms	Server (optional)
SpawnParamTuner	L2 (Params)	ResNet-MLP + bi-level gradient optimization	~325K	Synthetic (c, θ) pairs + real-game d_curve labels	N/A (offline)	<code>policies.json</code> → L1
ConvSharedPolicyValueNet	L3 (Placement)	CNN + DockBoard cross-attention + residual trunk + PPO/GAE + 7 auxiliary heads	~188K	Self-play rollouts (234K+ episodes)	<5 ms	Python server + browser

Model	Layer	Algorithm	Parameters	Training Data	Latency	Deployment
LightPolicyValueNet	L3 (Placement)	Dual-tower 2-layer MLP	~28K	Self-play (CPU)	<2 ms	Python CPU
LightSharedPolicyValueNet	L3 (Placement)	Single shared MLP + action embedding	~50K	Self-play (CPU)	<3 ms	Python CPU
Browser LinearAgent	L3 (Placement)	Linear softmax policy + value baseline + REINFORCE	~28K	Browser self-play	<1 ms	Browser (training panel)
MLX RL	L3 (Placement)	Lightweight policy-value net (mirrors PyTorch)	~28-50K	Self-play (MLX-accelerated)	<3 ms	Apple Silicon

Key design constraint: L1 (Spawn) and L3 (Placement) are fully orthogonal. The spawn engine determines *which blocks to show*; the RL agent determines *where to place them*. They share only the board state encoding—never the decision logic.

7.2 SpawnPolicyRules: Rule-Based Heuristic Engine

SpawnPolicyRules is the default, always-available spawn path. It uses **zero learned parameters**—all weights, thresholds, and profiles are configured in `shared/game_rules.json`. Despite being rule-based, it implements sophisticated algorithmic components:

Component	Algorithm	Complexity
Priority scheduler	10-intent queue with configurable priority scores	O(1) lookup
Board perception	17-signal fusion with weighted scoring	O(n^2), n=8
Weight chain	14-dim weighted sampling from 28-shape catalog	O(
Constructive pre-scan	C1 (Completer) + C2 (Setup) + C3 (Order Anchor)	O(
Two-stage construction	Stage 1 (clearSeats) + Stage 2 (weightedFill) + PEOG clamp	O(
Constraint gate	Shape uniqueness (O(1)) → Mobility guard (O(	A
Retry loop	22 attempts with exponential backoff + fallback_simple	O(22 × pipeline)

The full 9-layer pipeline and detailed algorithmic descriptions are provided in §5.

7.3 SpawnPolicyNet: Transformer-Based Neural Spawn

SpawnPolicyNet learns the conditional distribution $P(s_1, s_2, s_3 \mid B, \pi, H)$ from real-world data, capturing patterns that hand-tuned weight chains cannot express. It serves as an **optional alternative** to the rule engine, not a replacement: its output passes through the same constraint validation gate as SpawnPolicyRules, and any failure triggers automatic fallback.

Architecture (V3.1, ~317K parameters):

```
Input: 5 heterogeneous sources
├─ board (8×8, 64-dim) ⊗ behaviorContext (72-dim) → state_token [B,1,128]
├─ target_difficulty (1-dim) → diff_token [B,1,128]
├─ playstyle_id (discrete) → style_token [B,1,128] (optional)
├─ history (3 rounds × 3 shape IDs) → hist_tokens [B,9,128]
└─ CLS (learnable) [B,1,128]
```


Tokens: [CLS, state, diff, style?, histo..histo] $\in \mathbb{R}^{B \times 13 \times 128}$

↓

TransformerEncoder ×6 layers

Multi-Head Self-Attention: 4 heads, $d_{\text{model}}=128$, Pre-LN

FFN: Linear(128→256)→GELU→Linear(256→128), dropout=0.1

Residual + LayerNorm after each sub-layer

↓

CLS output → $h_c \in \mathbb{R}^{B \times 128}$

↓

Autoregressive Slot Heads	Auxiliary Multi-Task Heads
head0: $h_c \rightarrow \text{Linear}(128 \rightarrow 64)$ → GELU → Linear(64→28) [P(s ₁ ctx)]	diversity: $h_c \rightarrow 128 \rightarrow 3 \times 7$ (CE) difficulty: $h_c \rightarrow 128 \rightarrow 1$ (L1) feasibility: $h_c \rightarrow 128 \rightarrow 28$ (BCE) style: $h_c \rightarrow 128 \rightarrow 5$ (CE)
head1: $[h_c; \text{emb}(s_1)] \rightarrow$ Linear(256→64)→Linear(64→28) [P(s ₂ s ₁ ,ctx)]	intent: $h_c \rightarrow 128 \rightarrow 7$ (CE)
head2: $[h_c; \text{emb}(s_1); \text{emb}(s_2)]$ → Linear(384→64)→Linear(64→28) [P(s ₃ s ₁ ,s ₂ ,ctx)]	

Training:

Composite loss with 8 terms:

$$L = 1.0 \cdot L_{\text{ceAR}} + 0.3 \cdot L_{\text{div}} + 0.5 \cdot L_{\text{anti}} + 0.1 \cdot L_{\text{diff}} + 0.4 \cdot L_{\text{feas}} + 0.2 \cdot L_{\text{softInfeas}} + 0.15 \cdot L_{\text{style}} + 0.10 \cdot L_{\text{intent}}$$

Data sources: (1) real player replays, (2) rule-engine synthetic games, (3) self-play rollouts, (4) offline distillation (rule teacher → neural student).

LoRA Personalization: $W_{\text{adapted}} = W_{\text{base}} + (\alpha/r) \cdot BA$, rank $r=4$, $\alpha=16$. Injected at self_attn.q_proj + v_proj + dim_feedforward. 5.6K parameters per player (~1.8% of trunk). Loading ~30ms on player switch.

Full details in §8.

7.4 SpawnParamTuner: ResNet-MLP Parameter Optimization

SpawnParamTuner learns to predict the optimal difficulty curve $D^{\otimes}$ given a player context c and parameter vector $\theta \in [0,1]^{\{36\}}$, then uses gradient-based search to find θ^*_c for each context.

Architecture (~325K parameters):

Input: context_embedding(32) $\oplus$ $\theta(36) = 68\text{-dim}$

↓

Linear(68→128) → LayerNorm → GELU

↓

ResBlock

×8:

each

=

[Linear(128→128)→LN→GELU→Dropout(0.1)→Linear(128→128)→LN→x→GELU]

↓

LayerNorm(128)

↓

Output Heads (7 heads, all from shared LayerNorm output)

head_curve: 128→64→20 (sigmoid) $D(r)$: difficulty over score

head_curve_e:	128→64→20	(sigmoid)	E(r): delight/engagement
head_curve_f:	128→64→20	(sigmoid)	F(r): frustration cap
head_pb:	128→64→1	(sigmoid)	PB break probability
head_noMove:	128→64→1	(sigmoid)	Normalized no-move steps
head_score:	128→64→1	(linear)	Predicted log score
head_survival:	128→64→1	(sigmoid)	Survival probability

Bi-level optimization:

Inner level (learn $f_\varphi: (c, \theta) \rightarrow D^{\otimes}$):

$\min_\varphi E_{(c, \theta) \sim D} [L_{\text{total}}(f_\varphi(c, \theta), \text{targets})]$

with 15-term composite loss including: shape MSE ($\alpha=5$), anchor hinge ($\kappa=1$, 22 key r-points), monotonicity ($\mu=1$), diversity ($\tau=1 \rightarrow 0.005$), deploy loss ($v=1$), smoothness ($\epsilon=0.04$), balance ($\beta=0.15$), surprise ($\gamma=0.3 \rightarrow \sim 7\%$).

Outer level (search for θ^*_c):

$\theta^*_c = \operatorname{argmin} \{\theta \in [0, 1]^{\{36\}} J(f_\varphi^*(c, \theta))\}$

with 8 LHS restarts, T=300 Adam steps ($\eta=0.05$), reprojection to $[0, 1]^{\{36\}}$.

Deployment: trained policies $\rightarrow$ `policies.json` $\rightarrow$ consumed by both `SpawnPolicyRules` and `SpawnPolicyNet`.

7.5 ConvSharedPolicyValueNet: Self-Play RL Placement Agent

The primary RL model for learning optimal block placement. Trained via self-play PPO + GAE with seven auxiliary supervision heads.

Architecture (~188K parameters):

```

Grid Encoder:
[B, 1, 8, 8]
  → Conv2d(1→32, 3×3)→GELU
  → ResConvBlock(32)
  → ResConvBlock(32)
  → [B, 32, 8, 8] → AvgPool → g [B, 32]
[B, 16]
[B, 48]

Dock Encoder:
dock_mask_k [B, 25]
  → Q_k = Linear(25→16)
grid_feat [B, 32, 8, 8]
  → K = Conv2d(1×1, 32→16)
  → V = Conv2d(1×1, 32→16)
  softmax(Q_k · K / √16) · V^T → ctx_k
dock_ctx = concat(ctx_1, ctx_2, ctx_3)

Shared Trunk:
concat[scalars(65), g(32), dock_ctx(48)] = [B, 145]
  → x + GELU(Linear(145→128)(x)) (×3 residual FC blocks)
  → h(s) [B, 128]

Heads:
Policy: concat[h(s), GELU(ActionProj(15→48)(ψ(a)))] → 176→64→1 → logit → softmax
Value: h(s) → 128→64→1 → V(s)
Auxiliary (7 heads, all from h(s) or concat[h(s), ψ(a)]):
  board_quality(1), feasibility(1), survival(1), topology(10),
  spawn_diff(12 with v13 placeability), hole(1), clear_pred(4)

```

Training: PPO + GAE. PPO clip $\epsilon=0.25$, 4 epochs. GAE $\lambda=0.85$, $\gamma=0.99$. Mixed value target: 50% GAE returns + 50% log-normalized outcome score. Adaptive entropy target with feedback control. Difficulty bucket

curriculum: SCD ceiling progressively relaxed from 0.3→1.0 over 30,000 episodes. BestGuard automatic rollback on evaluation regression.

Parameter breakdown: Grid encoder (~5K) + Dock attention (~3K) + Shared trunk (~50K) + Policy head (~15K) + Value head (~8K) + 7 auxiliary heads (~107K).

Full details in §7.3–7.7.

7.6 Lightweight RL Variants

LightPolicyValueNet (~28K parameters). Dual-tower 2-layer MLP: independent policy and value towers from the same state input. Policy: $s \rightarrow \text{Linear}(204 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow |A|)$. Value: $s \rightarrow \text{Linear}(204 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow 1)$. Suitable for CPU training and resource-constrained environments. Implements `_AuxStubsMixin` for interface compatibility with the full ConvShared architecture.

LightSharedPolicyValueNet (~50K parameters). Single shared MLP ($204 \rightarrow 64 \rightarrow 64$) with action embedding ($15 \rightarrow 32$). Fused policy head ($h(s) \parallel \text{action_embed} \rightarrow \text{logits}$) and independent value head. Better representational capacity than the dual-tower variant while remaining $\sim 4\times$ smaller than ConvShared.

Browser LinearAgent (~28K parameters). Fully client-side RL: linear softmax policy $\pi(a|s) = \text{softmax}(W \cdot s)$ + linear value baseline $V(s) = w \cdot s$. Trained via REINFORCE ($n_epochs=1$) in the browser. Strategy-aware via `strategy_id` conditioning. Supports remote RL workflow (local trajectory collection → server batch PPO update). Training panel provides real-time weight visualization.

MLX RL (Apple Silicon). Mirrors LightPolicyValueNet/LightSharedPolicyValueNet architectures using Apple’s MLX framework with Metal Performance Shaders acceleration. Enables training on Apple Silicon Macs without CUDA. Shares `features.py` with PyTorch RL for cross-framework feature parity.

7. The RL Agent: Self-Play Placement Policy

7.1 Problem Formulation

The placement problem is formalized as a finite-horizon MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T)$:

- $\mathcal{S} \subset \mathbb{R}^{204}$: the 204-dimensional state feature vector (§7.2).
- $\mathcal{A}_t$: the set of legal placement actions at step t , with $|\mathcal{A}_t| \in [0, 192]$.
- $\mathcal{P}$: deterministic transition (placement → clear → new dock → next state).
- $\mathcal{R}$: reward function:

$$r_t = \text{\textcolor{red}{Deltascore}} + 0.8 \cdot \Delta \Phi_{\text{topology}} + 0.6 \cdot r_{\text{eval}} + 35 \cdot 1[\text{score} \geq \text{threshold}]$$

The potential function Φ shapes the reward without changing the optimal policy (Ng 1999):

$$\Phi(B) = -0.4 \cdot \text{holes} - 0.08 \cdot \text{transitions} - 0.15 \cdot \text{wells} + 0.35 \cdot \text{closeToFull} + 0.12 \cdot \text{mobility}$$

The evaluation feedback term r_{eval} is an instantaneous reward (not a potential difference, so it doesn’t create spurious energy):

$$r_{\text{eval}} = -0.10 \cdot \text{regretClipped} + 0.05 \cdot \text{optimality} - 0.08 \cdot \text{forcedBad} + 0.04 \cdot \text{salvage}$$

The agent’s objective:

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

with $\gamma = 0.99$ and termination when no dock block has a legal placement.

7.2 State and Action Feature Encoding

State vector $\in \mathbb{R}^{204} = 65\text{scalars} + 64\text{grid} + 75\text{dock}$:

Scalar segment (65 dimensions):

Sub-vector	Dim	Content
Structural primitives	25	fill_ratio, row/col max/min/mean/std, almost_full row/col ratios, unplaced_ratio, hole_ratio (normalized by 16), row_trans (norm by 64), col_trans (norm by 64), wells (norm by 24), close1/n (norm by 8), close2/n (norm by 8), mobility (norm by 192), height_std (raw), contiguous_regions (norm by 16), concave_corners (norm by 32)
Color summary	19	8 color occupancy ratios + 8 single-color-line potentials + 3 dock slot colors
Spawn step difficulty	4	scdNorm, comboCellsNorm, comboKillerNorm, comboLongBarNorm
Spatial planning	3	regionEntropy, largestRegionRatio, smallRegionCellRatio
Strategy one-hot	3	[easy, normal, hard]
Condition tokens	11	Arc: [opener, momentum, peak, fatigue, cooldown] + Intent: [relief, engage, pressure, flow, harvest, maintain]

Action feature $\psi(a) \in \mathbb{R}^{15}$:

- `nearFullRatio`: proportion of grid cells adjacent to the shape footprint that are near-completion.
- 8 neighbor features: for each of the 8 adjacent cells (up, down, left, right, 4 corners), a binary indicator of whether placing the shape would fill an empty cell adjacent to an occupied cell (capturing “edge adherence”).
- 6 self-features: shape aspect ratio (w/h), cell count, is_line, is_square, is_large (≥ 5 cells), category one-hot index.

7.3 Network Architecture: ConvSharedPolicyValueNet

The ConvSharedPolicyValueNet (v5, ~188K parameters at width=128, conv_channels=32) uses a shared trunk with specialized encoders for each input modality:

Grid Encoder:

$$\begin{aligned}
g_0 &= GELU(Conv2d(B_{embed}, 1 \rightarrow 32, 3 \times 3, pad = 1)) \\
g_1 &= ResConvBlock(g_0) = g_0 + GELU(Conv2d(GELU(Conv2d(g_0)))) \\
g_2 &= ResConvBlock(g_1) \\
g_{pooled} &= \frac{1}{64} \sum_{i,j} g_2[:, :, i, j] \in \mathbb{R}^{32}
\end{aligned}$$

DockBoard Attention. Each dock block attends to the CNN grid features before pooling:

$$\begin{aligned}
Q_k &= W_q \cdot mask_k \in \mathbb{R}^{16} \quad (k = 1, 2, 3 \text{dockslots}) \\
K &= Conv2d_{1 \times 1}^{32 \rightarrow 16}(g_2) \in \mathbb{R}^{16 \times 8 \times 8} \\
V &= Conv2d_{1 \times 1}^{32 \rightarrow 16}(g_2) \in \mathbb{R}^{16 \times 8 \times 8}
\end{aligned}$$

For each dock block k , the attention output is:

$$ctx_k = softmax \left(\frac{Q_k \cdot K_{flattened}}{\sqrt{16}} \right) \cdot V_{flattened}^T \in \mathbb{R}^{16}$$

The final dock context is $Linear_{16 \rightarrow 16}(ctx_k)$ for each k , flattened to $\mathbb{R}^{48}$.

Shared Trunk:

$$\begin{aligned}
x_0 &= [\text{scalars}, g_{\text{pooled}}, \text{dockCtx}] \in \mathbb{R}^{65+32+48=145} \\
x_1 &= x_0 + \text{GELU}(\text{Linear}_{145 \rightarrow 128}(x_0)) \\
x_2 &= x_1 + \text{GELU}(\text{Linear}_{128 \rightarrow 128}(x_1)) \\
h(s) &= x_2 + \text{GELU}(\text{Linear}_{128 \rightarrow 128}(x_2)) \in \mathbb{R}^{128}
\end{aligned}$$

Output heads:

- **Policy:** $h(s) \parallel \text{GELU}(\text{actionProj}_{15 \rightarrow 48}(\psi(a))) \rightarrow \text{Linear}_{176 \rightarrow 64} \rightarrow \text{GELU} \rightarrow \text{Linear}_{64 \rightarrow 1} \rightarrow \text{logit}(a)$. Logits are masked to legal actions and softmax-normalized.
- **Value:** $h(s) \rightarrow \text{Linear}_{128 \rightarrow 64} \rightarrow \text{GELU} \rightarrow \text{Linear}_{64 \rightarrow 1} \rightarrow V(s)$.

7.4 Training Algorithm

PPO objective:

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_t [\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) A_t)]$$

where $\rho_t = \pi_{\text{new}}(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$ and $\varepsilon = 0.25$.

GAE advantage estimation:

$$A_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

with $\lambda = 0.85$, $\gamma = 0.99$.

Mixed value target. The value head learns a hybrid target combining sparse outcome signal (low variance, no credit assignment problem) with dense GAE returns (temporal credit assignment):

$$R_t = 0.5 \cdot \text{GAE}_t + 0.5 \cdot \text{clip}\left(\frac{\log(1 + \text{finalScore})}{\log(1 + \text{winThreshold})}, 0, 3\right)$$

The log-normalized outcome target compresses the wide range of possible scores (0–50,000+) into a bounded [0, 3] range, preventing the value loss from being dominated by long-game returns.

Value loss (double-clipped SmoothL1):

$$\begin{aligned}
v_{\text{clipped}} &= v_{\text{old}} + \text{clamp}(v_{\text{new}} - v_{\text{old}}, -0.25, +0.25) \\
\mathcal{L}_{\text{value}} &= \mathbb{E} [\max(\text{SmoothL1}(v_{\text{new}}, R_t, \beta = 10), \text{SmoothL1}(v_{\text{clipped}}, R_t, \beta = 10))]
\end{aligned}$$

where $\text{SmoothL1}(x, y, \beta) = \begin{cases} 0.5(x - y)^2 / \beta & |x - y| < \beta \\ |x - y| - 0.5\beta & \text{otherwise} \end{cases}$ with $\beta = 10.0$ (Huber loss with larger quadratic region than standard $\beta = 1.0$).

Advantage normalization:

$$A_t^{\text{norm}} = \text{clamp}_{[-30, 30]} \left(\frac{A_t - \mu_A}{\max(\sigma_A, 10^{-4})} \right)$$

The ± 30 clamp and 10^{-4} minimum standard deviation guard prevent numerical instability from low-variance advantage batches.

Entropy loss (negative sign = maximizing entropy):

$$\mathcal{L}_{\text{entropy}} = -w_e \cdot \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \pi(a | s) \log \pi(a | s)$$

with w_e starting at 0.025 and linearly annealing to 0.008 over the first 50,000 episodes.

7.5 Auxiliary Supervision Heads

Seven auxiliary heads provide dense, per-step gradient signals independent of sparse Monte Carlo returns:

Head	Dim	Loss	Target	Coef	Correlation with Score ®
board_quality	1	SmoothL1($\beta=1$)	$\Phi(s) / 30$	0.5	+0.011 (p=0.86)
feasibility	1	BCE(logits)	DFS sequential solvability	0.3	-0.172 (p<0.0001)
survival	1	SmoothL1($\beta=1$)	$T_{remaining}/30$	0.2	-0.202 (p<0.0001)
topology_aux	10	SmoothL1($\beta=1$)	Post-placement topology vector	0.0	—
spawn_diff_aux	12	SmoothL1($\beta=1$)	4-dim SCD + 8-dim placeability	0.05	-0.009 (p=0.69)
hole_aux	1	SmoothL1($\beta=1$)	Unfillable cells / 16	0.0	—
clear_pred	4	CrossEntropy	Clear category $\{0,1,2,\geq 3\}$	0.15	—

The coefficients `hole_aux=0` and `topology_aux=0` indicate these heads are implemented but disabled in the current configuration; they can be activated by setting their environment variable overrides.

Feasibility head architecture:

$$feasLogit = Linear_{128 \rightarrow 64} \rightarrow GELU \rightarrow Linear_{64 \rightarrow 1}(h(s))$$

Logits are clamped to ± 10 before BCE computation to prevent numerical explosion ($\sigma(\pm 10) \approx 0/1$ already saturates).

Spawn diff aux head architecture:

$$sdPred_{12} = Linear_{128 \rightarrow 64} \rightarrow GELU \rightarrow Linear_{64 \rightarrow 12}(h(s))$$

Total auxiliary loss:

$$\mathcal{L}_{aux} = \sum_k w_k \cdot clamp_{[-20,20]}(\mathcal{L}_k)$$

The ± 20 hard clamp on auxiliary losses (but not on policy/value losses) prevents occasional numerical explosions from extreme board states (historically observed: `loss_bq` reaching 936,449, `loss_feas` reaching $\pm 7.8 \times 10^5$). These explosions were traced to the auxiliary head outputs diverging to $\pm 10^3 - -10^5$ on extreme boards, and have been mitigated by per-head prediction clipping (`board_quality` preds clamped to ± 10 , `survival` to ± 3 , `feasibility` logits to ± 10) in addition to the loss-level clamp.

7.6 Exploration and Curriculum

Temperature-softened policy with Dirichlet exploration:

$$\pi_{sample}(a|s) = 0.92 \cdot softmax(logits/T_t) + 0.08 \cdot Dirichlet(0.28, \dots, 0.28)$$

Temperature schedule: $T_t = 1.2$ for the first 2 moves of each episode (encouraging exploration), decaying to $T_t = 0.6$ thereafter (exploiting learned patterns).

Adaptive entropy target. An entropy target band (0.2 width around a configurable center) uses feedback control: if the batch entropy mean exceeds the target, w_e is reduced by 10% to discourage excess randomness; if entropy falls below, w_e is increased by 10%.

Difficulty bucket curriculum. Training episodes are gated by a progressive `maxScd` ceiling:

Stage	Episodes	maxScd	Allowed Buckets
1	0–5,000	0.3	Trivial, Easy
2	5,001–15,000	0.5	+Standard
3	15,001–30,000	0.7	+Hard
4	30,000+	1.0	All (full difficulty)

Docks whose SCD exceeds the current stage's `maxScd` are rejected and the spawn engine retries with a lower difficulty target (up to `retryCap = 6` attempts). This prevents early training from being dominated by impossible board states.

7.7 Training Infrastructure

The training system uses a multi-process architecture for maximum throughput:

```
Main Process (GPU)
├─ Parameter update (Adam, lr=3e-4 → linear warmup → cosine decay)
├─ BestGuard (evaluation-based rollback protection)
├─ Checkpoint serialization (every N episodes)
└─ Version counter increment (triggers worker reload)

Worker Pool (CPU, N workers)
├─ Worker 1: load weights(v) → inference_mode → collect K episodes → return trajectories
├─ Worker 2: ... (independent, parallel)
├─ ...
└─ Worker N: ...
```

Key design decisions:

- **Worker inference with `torch.inference_mode()`** : Drops autograd tracking, view tracking, and version checking—reducing per-step overhead by ~20% compared to `torch.no_grad()`.
- **Weight broadcast via tempfile**: Workers reload weights from disk when the version counter increments. This avoids IPC serialization latency (model is ~900KB → ~2ms disk read).
- **Single-threaded workers**: Each worker forces `torch.set_num_threads(1)` to prevent N workers × M threads CPU oversubscription. The 188K-parameter network performs single-sample inference in <0.5ms; multi-threading offers no benefit and causes cache thrashing.
- **Batch collection**: 8 episodes per batch (configurable), PPO epochs = 4, mini-batch shuffling.
- **BestGuard**: Maintains a reference network snapshot and an evaluation gate. Every `eval_gate_every` episodes, the current network plays `eval_gate_games` evaluation games. If win rate drops below `eval_gate_win_ratio`, the model is rolled back to the best-ever checkpoint. This prevents catastrophic forgetting during long training runs. The guard also tracks teacher coverage, score moving average, and spawn difficulty drift.

8. Neural Spawn Generation: SpawnPolicyNet

8.1 Motivation and Design Goals

The rule-based spawn engine, while robust, has an inherent ceiling: it can only express designer-specified heuristics encoded in the 14-dimensional weight chain. Real player data contains distributional patterns that hand-tuned weights cannot capture—combinations of board topology, player state, and recent history that correlate with satisfying gameplay experiences but are too subtle or too numerous to encode explicitly.

SpawnPolicyNet learns $P(s_1, s_2, s_3 \mid B, \pi, H)$ directly from data: real player replays (ground-truth human preferences), rule-engine synthetic games (positive examples of rule-track behavior), and self-play rollouts (optimal-placement examples). It serves as an optional alternative, not a replacement: its output passes through exactly the same constraint validation gate as the rule track, and any validation failure triggers automatic, transparent fallback to the rule engine.

8.2 Model Architecture (V3.1)

SpawnPolicyNet V3.1 (~317K parameters) uses a Transformer encoder with three separate autoregressive slot decoder heads:

Input encoding:

$$\begin{aligned} stateToken &= LayerNorm(GELU(Linear_{88 \rightarrow 128}([B_{flat}; \pi]))) \in \mathbb{R}^{B \times 1 \times 128} \\ diffToken &= LayerNorm(GELU(Linear_{1 \rightarrow 128}(d))) \\ histTokens &= shapeEmbed_{29 \times 128}[H_{ids}] + posEmbed_{9 \times 128} \in \mathbb{R}^{B \times 9 \times 128} \\ clsToken &= trainableParam \in \mathbb{R}^{1 \times 128} \end{aligned}$$

Sequence: $tokens = [cls, state, diff, hist_0, \dots, hist_8] \in \mathbb{R}^{B \times 12 \times 128}$

Encoder: 6-layer TransformerEncoder ($d_{model} = 128$, $n_{heads} = 4$, $FFNDim = 256$, GELU, dropout=0.1, norm_first=True).

Slot heads (autoregressive):

$$\begin{aligned} CLS_{out} &= LayerNorm(encoded[:, 0, :]) \in \mathbb{R}^{B \times 128} \\ l_0 &= Linear_{128 \rightarrow 28}(CLS_{out}) \\ l_1 &= Linear_{256 \rightarrow 28}(concat[CLS_{out}, emb(s_1)]) \\ l_2 &= Linear_{384 \rightarrow 28}(concat[CLS_{out}, emb(s_1), emb(s_2)]) \end{aligned}$$

The progressive dimension increase (128→256→384) reflects the growing context: head0 sees only the shared encoding; head1 additionally sees the first slot's embedding; head2 sees both preceding slots.

Auxiliary heads (from CLS token):

Head	Output	Loss	Purpose
diversity	$\mathbb{R}^{B \times 3 \times 7}$	Cross-entropy	Predict category distribution of each slot
difficulty	$\mathbb{R}^{B \times 1}$	SmoothL1	Align difficulty prediction with target
feasibility	$\mathbb{R}^{B \times 28}$	BCE (per-shape)	Predict which shapes are placeable
style	$\mathbb{R}^{B \times N_{styles}}$	Cross-entropy	Style self-supervision
intent	$\mathbb{R}^{B \times N_{intents}}$	Cross-entropy	Intent self-supervision

8.3 Training

V3.1 composite loss:

$$\mathcal{L}_{V3.1} = 1.0\mathcal{L}_{ce-AR} + 0.3\mathcal{L}_{div} + 0.5\mathcal{L}_{anti} + 0.1\mathcal{L}_{diff} + 0.4\mathcal{L}_{feas} + 0.2\mathcal{L}_{si} + 0.15\mathcal{L}_{style} + 0.10\mathcal{L}_{intent}$$

Where:

- $\mathcal{L}_{ce-AR}$: $-\frac{1}{3} \sum_{k=1}^3 \log P(s_k | s_{<k}, ctx)$ (teacher forcing).
- $\mathcal{L}_{div}$: Category distribution entropy maximization.
- $\mathcal{L}_{anti}$: Penalty on repeated shapes or same-family shapes.
- $\mathcal{L}_{feas}$: $-\frac{1}{28} \sum_{j=1}^{28} [y_j \log \sigma(l_j) + (1 - y_j) \log(1 - \sigma(l_j))]$ (per-shape BCE).
- $\mathcal{L}_{si}$: Soft penalty on logits for infeasible shapes: $\frac{1}{|\mathcal{I}|} \sum_{j \in \mathcal{I}} \max(0, l_j - l_{max})$, where $\mathcal{I}$ is the set of infeasible shapes and l_{max} is the maximum logit among feasible shapes.
- $\mathcal{L}_{style}, \mathcal{L}_{intent}$: Self-supervised cross-entropy on style and intent labels.

Data sources:

- Player replays** (): Real dock choices from human gameplay—the gold-standard distribution.
- Rule-engine synthetic** (): Generated by running `SpawnPolicyRules` over diverse simulated profiles—provides positive rule-track examples.
- Self-play rollouts** (): RL bot games—optimal-placement labeled examples.

4. **Offline distillation:** Rule-track teacher $\rightarrow$ neural student, ensuring the model can at least reproduce rule-track quality.

8.4 LoRA Personalization

$$W_{adapted} = W_{base} + \frac{\alpha}{r} \cdot BA$$

with $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$, rank $r = 4$, $\alpha = 16$. Injection points: `self_attn.q_proj` + `v_proj` in each encoder layer. Per-player parameters: 5.6K (~1.8% of trunk). Loading latency: ~30ms (one-time on player switch).

8.5 Inference and Safety

- **Latency:** 4–8ms CPU per forward pass.
- **Feasibility mask:** <0.05ms (28 calls to vectorized `get_legal_positions`).
- **Validation:** Output passes through the same constraint gate (§5.3 Layer 6).
- **Fallback:** On any gate failure or service unavailability, automatic transparent fallback to Track 1 (rule engine), with `fallbackReason` recorded in diagnostics.

9. Spawn Parameter Tuning: SpawnParamTuner

9.1 Problem Statement

SpawnPolicyRules behavior is governed by a 36-dimensional parameter vector $\theta \in [0, 1]^{36}$:

$\theta = [personalization_5, pbTension_4, scoring_8, translation_5, challenge_5, order_2, constructive_2, solution_2, special_3$

For each player context c (representing lifecycle $\times$ maturity $\times$ arc $\times$ PB bin), the goal is to find the optimal parameter vector θ_c^* that produces the ideal difficulty progression curve $D(r)$.

9.2 Bi-Level Optimization

Inner level (learn $f_\phi : (c, \theta) \rightarrow D(r)$):

$$\min_{\phi} \mathbb{E}_{(c, \theta) \sim \mathcal{D}} [\mathcal{L}_{total}(f_\phi(c, \theta), D_{target})]$$

Outer level (search for θ_c^*):

$$\theta_c^* = \arg \min_{\theta \in [0, 1]^{36}} \mathcal{J}(f_{\phi^*}(c, \theta))$$

with 8 LHS restarts, T=300 Adam steps at $\eta = 0.05$, reprojection to $[0, 1]^{36}$.

9.3 Deployment

Trained policies exported as `policies.json`: $\{c_1 : \theta_1^*, c_2 : \theta_2^*, \dots\}$. The 4 PB curve parameters are consumed by both `SpawnPolicyRules` and `SpawnPolicyNet`.

10. Monetization Framework

The monetization framework implements experience-first monetization:

Whale score:

$whale = 0.4 \cdot \min(1, bestScore/2000) + 0.3 \cdot \min(1, totalGames/50) + 0.3 \cdot \min(1, sessionMin/10)$

Segments: whale (≥ 0.60), dolphin ($[0.30, 0.60)$), minnow (< 0.30).

Decision engine: filter $\rightarrow$ render $\rightarrow$ sort $\rightarrow$ explain (4-step pipeline).

State-gated triggers: No ads during anxiety (frustration ≥ 3); no IAP during flow; cool-down ≥ 3 minutes between ads.

11. Engineering Infrastructure

11.1 Technology Stack

Layer	Technology	Version
Frontend	Vanilla JS (no framework) + Vite	ES2022
Rendering	Canvas 2D / WebGL	—
RL (PyTorch)	Python 3.10+, PyTorch 2.x, Numba JIT	—
RL (MLX)	Apple MLX	Apple Silicon
Backend	Python Flask	3.x
Database	SQLite (dev) / PostgreSQL (prod)	SQLite 3.x, PG 15+
Observability	Prometheus + OpenTelemetry (W3C tracecontext)	—
Deployment	Docker Compose + Kubernetes/Helm	—

11.2 Performance

- **Frontend bundle:** 500KB $\rightarrow$ 230KB (−54%) via Vite `manualChunks`.
- **RL simulator:** Numba JIT kernel for legal position enumeration ($\sim 0.02\text{ms}$ vs $\sim 0.2\text{ms}$ pure numpy).
- **DFS feasibility:** $\sim 50\text{ms}$ (easy board) to $\sim 200\text{ms}$ (hard board) in browser; $\sim 2\text{--}10\text{ms}$ with Numba in Python.
- **Per-shape placeability:** $\sim 0.16\text{ms}$ (8 `get_legal_positions` calls, Numba).

11.3 Architecture Decision Records

Ten ADRs document decisions from bitmap encoding (ADR-001) to WASM compilation (ADR-009).

11.4 Cross-Platform Contract

JS $\leftrightarrow$ Python parity enforced by shared fixture: `tests/fixtures/spawnStepDifficulty.cases.json`. 75 test cases cover edge conditions (empty board, full board, various fill ratios and shape distributions).

12. Evaluation and Empirical Results

12.1 RL Agent Training Performance

The agent was trained for 234,000+ self-play episodes with the balanced preset (batch_size=8, ppo_epochs=4, entropy_coef annealing 0.025 $\rightarrow$ 0.008, adaptive victory threshold). Analysis from the most recent 2,100 training

episodes:

Outcome distribution:

Metric	Value
Total episodes	2,100
Wins	748 (35.6%)
Losses	1,352 (64.4%)
Mean score (overall)	5,739
Median score	4,200
P25 score	1,795
P75 score	8,040
Maximum score	43,480
Mean steps (won games)	263.7
Mean steps (lost games)	67.3

Key insight: The winning vs. losing step count gap (264 vs. 67) is stark. Winning games are 4× longer than losing games, indicating that the primary failure mode is rapid board degeneration leading to unplaceable docks relatively early in the episode. This points directly to the long-bar bottleneck.

Loss component decomposition:

Loss Component	Raw Mean	Coefficient	Effective Contribution	r(score)	r(won)
loss_policy	0.06705	1.0	0.06705	—	—
loss_value	16.68	0.5	8.34	—	—
loss_feas (BCE)	0.03943	0.3	0.01183	−0.172***	−0.160***
loss_surv (MSE)	0.02148	0.2	0.00430	−0.202***	−0.170***
loss_bq (MSE)	0.00123	0.5	0.00062	+0.011	+0.004
loss_spawn_diff (pre-v13)	0.02417	0.05	0.00121	−0.009	−0.007
loss_topology	0.00350	0.0	0.0	—	—
Entropy	1.565	−0.01	−0.0157	—	—
approx_kl	0.255	—	—	—	—

*** $p < 0.0001$

The feasibility and survival auxiliary heads show statistically significant negative correlation with both score (Pearson $r = -0.172, -0.202$) and win/loss (point-biserial $r = -0.160, -0.170$). This means that episodes where the model is *worse at predicting* feasibility and survival (higher loss) are episodes that end in losses with lower scores. Conversely, the board quality head shows near-zero correlation with outcomes ($r = +0.011, p = 0.86$), suggesting that predicting board quality alone is insufficient for gameplay success.

12.2 Spawn Difficulty Distribution

Across all training episodes, the observed spawn step difficulty bucket distribution:

Bucket	Proportion
Standard (0.4–0.6)	59.3%
Hard (0.6–0.8)	32.8%

Bucket	Proportion
Extreme (>0.8)	5.9%
Easy ($0.2\text{--}0.4$)	2.0%
Trivial (<0.2)	0.02%
Mean SCD	0.585
Max SCD (per episode avg)	0.996

Critical finding: The bucket distribution is nearly identical between won and lost games:

Bucket	Won Games	Lost Games	Difference
Standard	59.4%	59.3%	+0.1%
Hard	32.8%	32.8%	0.0%
Extreme	5.9%	5.9%	0.0%
Easy	1.9%	2.1%	−0.2%

The per-episode mean SCD is 0.586 for won games and 0.585 for lost games—a difference of $<0.2\%$. This confirms that spawn difficulty distribution is not the differentiating factor between wins and losses. The system generates similarly difficult docks for both outcomes. The difference lies in the board state’s capacity to accommodate them: losing boards have degenerated to the point where even standard-difficulty docks become partially unplaceable, reducing the effective action space and triggering the death spiral.

12.3 Long-Bar Bottleneck Verification

Simulation experiment design: 2,000 random boards per fill level with 15% hole density (representative of real gameplay conditions). For each board, count legal positions for each shape in the 28-shape catalog and aggregate by category.

Fill Rate	Long-Bars ($1\times4, 4\times1, 1\times5, 5\times1$)	Square (2×2)	Non-Line Shapes	Gap	Long-Bar Zero-Pos. Rate
40%	0.998	0.999	0.999	−0.001	0.2%
50%	0.978	0.987	0.987	−0.009	2.2%
60%	0.893	0.952	0.944	−0.051	10.7%
65%	0.787	0.885	0.884	−0.096	21.3%
70%	0.671	0.800	0.798	−0.127	32.9%
75%	0.436	0.610	0.617	−0.181	56.4%
80%	0.302	0.458	0.468	−0.166	69.8%

Individual long-bar breakdown at 70% fill:

Shape	Placeability Rate	Mean Legal Positions	Zero-Position Rate
1×4	0.752	2.0	24.8%
4×1	0.921	2.4	7.9%
1×5	0.436	0.8	56.4%
5×1	0.576	1.0	42.4%

Horizontal long-bars (1×4 , 1×5) are significantly more vulnerable than vertical ones (4×1 , 5×1) because real-game boards tend to be filled from the bottom up, leaving more vertical than horizontal gaps. The 1×5 piece is the single most constrained shape at all fill levels above 50%.

12.4 v13 Per-Shape Placeability Analysis

The original 4-dimensional `spawn_diff_aux` head (`scd/cells/killer/longBar`) showed near-zero correlation with outcomes ($r = -0.009$, $p = 0.69$). The v13 extension adds 8 per-shape placeability dimensions. While training is ongoing at time of writing, the simulation data strongly supports the hypothesis that placeability signals will improve the trunk’s representation of board viability.

Computational cost analysis: The 8 `get_legal_positions` calls add $\sim 0.16\text{ms}$ per step (Numba JIT), compared to 50–200ms for the existing DFS feasibility check. Since the DFS check is already performed for the `feasibility` auxiliary target, the placeability computation effectively piggybacks on existing board analysis.

12.5 Cross-Language Contract Validation

All cross-language tests pass (75 test cases, shared fixture `spawnStepDifficulty.cases.json`). The JS and Python implementations of `spawn_step_difficulty_features`, `extract_state_features`, `board_potential`, and `fast_board_features` produce identical outputs to within floating-point tolerance ($1e-6$).

13. Related Work

Programmatic content generation. Togelius et al. (2011) established the search-based PCG taxonomy. OpenBlock’s constructive pre-scan (C1/C2/C3) implements a specialized local search over triplet $\times$ ordering configurations. Yannakakis & Togelius (2011) extended PCG to use player experience models as content evaluators—directly analogous to OpenBlock’s use of `PlayerProfile` as a real-time scoring function for spawn candidates.

Adaptive difficulty. Csikszentmihalyi (1990) formalized flow; Yerkes-Dodson (1908) the inverted-U arousal-performance curve. Hunicke (2005) demonstrated real-time DDA in Hamlet. Pasqualotto et al. (2024) showed multidimensional DDA with independent parameter control and probabilistic perturbation. OpenBlock extends these with a two-axis differentiation matrix (lifecycle $\times$ maturity) and a between-game difficulty arc (RoR) with cubic modulation.

Reinforcement learning for board games. AlphaGo Zero (Silver et al., 2017) and AlphaZero (Silver et al., 2018) demonstrated self-play RL with MCTS. OpenBlock applies similar architecture (policy+value network, self-play, search distillation) but for continuous content generation: the spawn difficulty serves as a curriculum, and evaluation gates prevent regression. The outcome-value mixing (GAE + log-normalized score) addresses the challenge of dense rewards with unbounded magnitude—a problem absent in binary-win/loss games.

Player modeling. Missura & Gärtner (2009) established dynamic difficulty with explicit player models. Conati et al. used Bayesian student modeling. OpenBlock’s EMA-based profiling prioritizes browser-compatible computation over modeling fidelity—a pragmatic trade-off.

Transformer content generation. Vaswani et al. (2017) introduced the Transformer. SpawnPolicyNet applies autoregressive triplet prediction with feasibility masking and LoRA personalization (Hu et al., 2021). The key architectural innovation is the progressive slot masking (CLS, CLS+s1, CLS+s1+s2), which maintains joint distribution consistency without an explicit autoregressive sequence model.

Commercial systems. Block Blast (Hungry Studio, 2022) and Color Block Jam (2025) are the primary commercial references. Neither has published its content generation architecture. Tetris (Pajitnov, 1984) established the “random piece generator” baseline, later refined with the 7-bag system. OpenBlock’s nine-layer pipeline goes substantially beyond simple random or bag-based generation.

14. Limitations and Future Work

14.1 Current Limitations

- **Board size generalization:** The 8×8 grid assumption is baked into feature encoding dimensions (64 grid cells), normalization constants (maxHoles=16, maxMobility=192), and per-shape placeability norms (theoretical maxima assume 8×8).
- **Shape pool expansion:** SpawnPolicyNet’s 28-shape vocabulary is fixed in the output head dimensions.
- **Heuristic player model:** All profiling uses hand-designed formulas, which may miss subtle behavioral signals.
- **MCTS search depth:** Browser-based MCTS limited to ≤ 3 –4 ply by CPU budget.
- **Monetization-RL separation:** Monetization signals are not integrated into RL reward shaping.
- **Single-player only:** No multiplayer or social features.

14.2 Future Directions

1. **Federated LoRA.** Train player-specific weights on-device; upload aggregate statistics only.
2. **LLM explainability.** Natural-language diagnostic explanations from spawn decision traces.
3. **Causal player modeling.** Counterfactual reasoning for parameter selection.
4. **Unified multi-objective RL.** Single reward function integrating spawn quality, placement quality, and monetization.
5. **Procedural level generation.** Sequences of docks with designed difficulty arcs, curated for specific skill-development goals.

15. Appendix

A. Glossary

Term	Definition
Spawn	Generation of the dock candidate triplet at each step
Dock	3 candidate shapes simultaneously visible to the player
SCD	Spawn Step Difficulty: 0–1 composite score of dock difficulty
DFS Solvability	Bounded depth-first search verifying sequential placeability
Flow	Psychological state where perceived challenge $\approx$ perceived skill
RoR	Rate of Return: between-game difficulty progression arc
25-Grid	Lifecycle (S0–S4) $\times$ Maturity (M0–M4) differentiation matrix
LoRA	Low-Rank Adaptation: parameter-efficient model personalization
GAE	Generalized Advantage Estimation (Schulman et al., 2015)
PPO	Proximal Policy Optimization (Schulman et al., 2017)

B. Key Configuration Parameters

Parameter	Default	Location	Description
<code>single_line</code>	20	<code>game_rules.json</code> <code>clearScoring</code> →	Base score unit
<code>perfectClearMult</code>	10	<code>game_rules.json</code> <code>clearScoring</code> →	Score multiplier for board wipe
<code>iconBonusLineMult</code>	5	<code>game_rules.json</code> <code>clearScoring</code> →	Multiplier for icon-matched lines
<code>comboMultiplier.activationStreak</code>	3	<code>game_rules.json</code> <code>clearScoring</code> →	Clears needed to activate combo
<code>rlRewardShaping.boardQualityLossCoef</code>	0.5	<code>game_rules.json</code> <code>rlRewardShaping</code> →	Weight for board quality aux loss
<code>rlRewardShaping.feasibilityLossCoef</code>	0.3	<code>game_rules.json</code> <code>rlRewardShaping</code> →	Weight for feasibility BCE loss
<code>rlRewardShaping.spawnDiffAux.coef</code>	0.05	<code>game_rules.json</code> <code>rlRewardShaping</code> →	Weight for spawn diff aux loss
<code>rlRewardShaping.spawnDiffAux.dim</code>	12	<code>game_rules.json</code> <code>rlRewardShaping</code> →	Spawn diff aux output dimension
<code>ppo_clip</code>	0.25	<code>train.py</code>	PPO ratio clipping epsilon
<code>gae_lambda</code>	0.85	<code>train.py</code>	GAE trace decay parameter
<code>gamma</code>	0.99	<code>train.py</code>	MDP discount factor
<code>MAX_SPAWN_ATTEMPTS</code>	22	<code>blockSpawn.js</code>	Retry budget for spawn generation

C. API Reference

Endpoint	Method	Description
<code>/api/rl/select_action</code>	POST	RL placement inference: accepts state features, returns action logits
<code>/api/spawn-model/v3/predict</code>	POST	Neural spawn: accepts board+context, returns dock triplet
<code>/api/spawn-model/v3/personalize</code>	POST	LoRA training: accepts player data, returns LoRA weights
<code>/api/evaluation/session</code>	GET	Returns <code>sessionEvalRecord</code> for a completed game session
<code>/api/rl/train_episode</code>	POST	Online PPO training: accepts trajectory, returns gradients

D. Reproduction Checklist

Environment: Python 3.10+, Node 20+, PyTorch 2.x, Numba 0.58+.

```
# Install dependencies
pip install -r requirements-rl.txt
npm install

# Run RL training (CPU, single process)
python -m rl_pytorch.train --device cpu --arch conv-shared --batch-episodes 256

# Run spawn model training
```

```
python -m rl_pytorch.spawn_model.train_v3

# Run cross-language contract tests
npx vitest run tests/spawnStepDifficulty.test.js
python -m pytest tests/test_spawn_step_difficulty.py -v

# Start dev server with full features
npm run dev
```

E. References

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
2. Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv:1506.02438*.
3. Silver, D., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144.
4. Silver, D., et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550, 354–359.
5. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*, 30.
6. Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*.
7. Csikszentmihalyi, M. (1990). *Flow: The Psychology of Optimal Experience*. Harper & Row.
8. Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *ICML*.
9. Yannakakis, G. N., & Togelius, J. (2011). Experience-driven procedural content generation. *IEEE Trans. Affective Computing*, 2(3), 147–161.
10. Togelius, J., Yannakakis, G. N., Stanley, K. O., & Browne, C. (2011). Search-based procedural content generation: A taxonomy and survey. *IEEE Trans. Computational Intelligence and AI in Games*, 3(3), 172–186.
11. Hunicke, R. (2005). The case for dynamic difficulty adjustment in games. *ACE 2005*.
12. Pasqualotto, A., et al. (2024). Multidimensional DDA in Legends of Hoa’Manu. EPFL/UNIGE Technical Report.
13. Missura, O., & Gärtner, T. (2009). Player modeling for intelligent difficulty adjustment. *Discovery Science*.
14. Schrittwieser, J., et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588, 604–609.

Document version: v1.1 | **Last updated:** 2026-07-01 | **License:** MIT

This report synthesizes material from the OpenBlock documentation tree. The authoritative, versioned source for each subsystem resides in the linked documents. This report provides a unified narrative suitable for researchers, engineers, and reviewers.